

DEPARTEMENT TECHNOLOGIES DE L'INFORMATIQUE

N° 2021/

MEMOIRE DE STAGE DE FIN D'ETUDES

POUR L'OBTENTION DU DIPLOME DE MASTERE PROFESSIONNEL
EN TECHNOLOGIES DE L'INFORMATIQUE :

GÉNIE LOGICIEL ET DÉVELOPPEMENT RAPIDE D'APPLICATIONS

-ENSEIGNEMENT A DISTANCE-

Intitulé :

Développement d'un système de collecte de
données en temps réel et de gestion des alertes
pour les machines en salle de réveil

Encadreur :

Pr. Ridha Azizi

Elaboré par :

Zaibi Racha

Intitulé du stage : Stage fin d'étude du master en GLDRA

Réalisé par les étudiants : Zaibi Racha

Organisme d'accueil : Faculté de médecine de Monastir

Mots-Clés : Surveillance médicale, salle de réveil, collecte de données en temps réel, OCR, YOLO, IoT médical, alertes médicales, notifications instantanées, Laravel, Flask, React, Flutter, méthodologie Scrum.

Résumé :

Ce mémoire présente la conception et le développement de VitalSync, un système innovant de collecte de données en temps réel et de gestion des alertes pour la surveillance des patients en salle de réveil. Réalisé au sein du SmartLab de la Faculté de Médecine de Monastir, ce projet répond aux contraintes des hôpitaux tunisiens, notamment l'hétérogénéité des équipements médicaux et le manque d'automatisation.

La solution combine des technologies de vision par ordinateur (YOLO, OpenCV, Tesseract OCR) et des outils de développement modernes (Laravel, Flask, React, Flutter) pour extraire les paramètres vitaux depuis les moniteurs, générer des alertes en cas d'anomalie et notifier en temps réel le personnel médical.

Le développement s'appuie sur la méthodologie Scrum, garantissant un processus itératif, collaboratif et adapté aux exigences du domaine médical. Les résultats obtenus montrent la faisabilité d'une solution économique, évolutive et compatible avec les équipements existants, contribuant à l'amélioration de la qualité des soins et à la sécurité des patients.

**Nom de l'Enseignant
Encadrant :**

Pr. Ridha Azizi

Signature :

**Nom de l'Encadrant
Industriel :**

Pr. Charffeddin
Ammeri

**Tampon & Sign.
de l'Encadrant
Ind. :**

Table des matières

Introduction Générale	11
1 Cadre Général du Projet	12
1.1 Introduction	12
1.2 Présentation de l'organisme d'accueil	12
1.3 Contexte et problématique	13
1.3.1 Variabilité des signes vitaux	13
1.3.2 Défis structurels et opérationnels	13
1.4 Analyse des solutions existantes	14
1.4.1 Présentation des solutions existantes	14
1.4.2 Comparaison et limites	15
1.5 Solution proposée : VitalSync	16
1.5.1 Fonctionnement de VitalSync	16
1.5.2 Les Avantages de VitalSync	17
1.6 Conclusion	17
2 Méthodologie et spécifications des besoins	18
2.1 Introduction	18
2.2 Analyse et spécification des besoins	18
2.2.1 Identification des acteurs	18
2.2.2 Spécification des besoins	19
2.3 Choix de la méthodologie	20
2.3.1 Etude comparative des méthodologies	20
2.3.2 Méthodologie retenue : Scrum	20
2.4 Présentation de Scrum	21
2.4.1 Artefacts Scrum et leur application	22
2.4.2 Événements Scrum et leur implémentation	22

2.5	Les rôles Scrum	23
2.6	Outils de support à Scrum	24
2.7	Conclusion	25
3	Étude architecturale du projet	26
3.1	Introduction	26
3.2	Présentation de l'architecture VitalSync	26
3.3	Modèle de conception logicielle adoptée	27
3.4	Interactions et fonctionnement du système	28
3.4.1	Diagramme des cas d'utilisation	28
3.4.2	Diagramme de classes	29
3.4.3	Diagramme de séquence	31
3.4.4	Diagramme d'activités	31
3.4.5	Diagramme de composants	32
3.4.6	Diagramme de déploiement	33
3.5	Pourquoi ce choix d'architecture	33
3.6	Conclusion	34
4	Sprint 0 : Planification et préparation du projet	35
4.1	Introduction	35
4.2	Pilotage du projet avec Scrum	35
4.2.1	Acteurs Scrum et Missions	35
4.2.2	Organisation du Backlog	36
4.2.3	Planification des sprints	37
4.3	Environnement de travail	38
4.3.1	Environnement matériel	38
4.3.2	Environnement de développement	39
4.3.3	Environnement logiciel	40
4.4	Conclusion	41
5	Sprint 1 : Création du dataset et mise en place du pipeline OCR	43

5.1	Introduction	43
5.2	Objectifs du Sprint 1	43
5.3	Tâches réalisées	45
5.3.1	Configuration du matériel	45
5.3.2	Capture et annotation des images	46
5.3.3	Entraînement du modèle de détection	46
5.3.4	Développement du pipeline OCR	47
5.3.5	Tests d'intégration et déploiement initial	48
5.4	Livrables du Sprint 1	48
5.5	Tests et validation	49
5.6	Défis et solutions	49
5.7	Planification pour le sprint 2	50
5.8	Conclusion	50
6	Sprint 2 : Système d'alertes et de notifications	51
6.1	Introduction	51
6.2	Objectifs	51
6.3	Activités	52
6.4	Résultats	55
6.5	Tests	56
6.6	Défis	56
6.7	Planification pour le sprint 3	57
6.8	Conclusion	57
7	Sprint 3 : Finalisation du système VitalSync	58
7.1	Introduction	58
7.2	Objectifs	58
7.3	Amélioration du tableau de bord	58
7.4	Interfaces mobiles	60
7.5	Conclusion	63

Conclusion générale et perspectives	64
Bibliographie	66

Table des figures

1.1	Logo de la FMM	12
1.2	Logo du SmartLab	12
1.3	Logo de Mindray	14
1.4	Logo de IQ Messenger	14
1.5	Logo de ChartX	15
2.1	Processus Scrum.	21
2.2	Rôles Scrum.	24
2.3	Logo de ClickUp.	24
2.4	Logo de Slack.	25
2.5	Logo de Git.	25
3.1	Architecture générale du système VitalSync	27
3.2	Modèle MVC de VitalSync	28
3.3	Diagramme des cas d'utilisation global	29
3.4	Diagramme de classes	30
3.5	Diagramme de séquence global	31
3.6	Diagramme d'activité du flux de données	32
3.7	Diagramme de composants du système	33
3.8	Diagramme de déploiement	33
4.1	Logos des technologies backend : Laravel et PHP	39
4.2	Logos des technologies utilisées pour le traitement et l'API Python : Python et Flask	39
4.3	Logos des technologies utilisées pour le développement mobile : Flutter et Dart	39
4.4	Logos des technologies utilisées pour le développement web : React et JavaScript	40
4.5	Logos des outils de gestion de base de données : MySQL et phpMyAdmin	40
4.6	Logo de Visual Studio Code	40

4.7	Logo d'Android Studio	41
4.8	Logo de LaTeX	41
4.9	Logo de GitHub	41
4.10	Logo de Docker Desktop	41
5.1	Architecture du système pour le Sprint 1	43
5.2	Courbe d'apprentissage du modèle de détection	47
5.3	Capture d'écran de l'interface Swagger UI pour la documentation de l'API REST	47
5.4	Diagramme de séquence du workflow du pipeline OCR	48
6.1	Diagramme de séquence pour la génération d'alertes et la distribution des notifications	53
6.2	Diagramme d'activité pour le processus de gestion des alertes	54
6.3	Diagramme des cas d'utilisation pour le sprint 2	55
7.1	Tableau de bord principal affichant les paramètres vitaux et les alertes.	59
7.2	Liste des patients avec options de filtrage.	59
7.3	Liste du personnel médical.	60
7.4	Ajouter un commentaire	61
7.5	Autoriser les notifications	61
7.6	Liste détaillée des patients	61
7.7	Accueil	61
7.8	Page d'accueil	61
7.9	Connexion	61
7.10	Liste des machines	62
7.11	Détails machine	62
7.12	Notifications	62
7.13	Patients par lit	62
7.14	Détails patient	62
7.15	Données machine du patient	62
7.16	Profil utilisateur	63

7.17	Notifications vues	63
7.18	Calendrier des shifts	63

Liste des tableaux

1.1	Plages normales des signes vitaux selon l'âge et le contexte	13
1.2	Comparaison des solutions de surveillance	15
1.3	Domaines couverts par VitalSync	17
2.1	Acteurs du système et leurs rôles	18
2.2	tableau comparatif des méthodologies de développement logiciel.	20
4.1	Acteurs du projet Scrum	36
4.2	Backlog du Projet avec Priorités MoSCoW	37
4.3	Planification des Sprints du Projet VitalSync	38
5.1	Sprint Backlog	45
5.2	Précision par classe pour le modèle de détection sur le jeu de validation . .	46
5.3	Résultats des tests du Sprint 1	49

Introduction Générale

Dans un contexte médical en constante évolution, l'optimisation de la surveillance des patients en salle de réveil constitue un enjeu crucial pour assurer des soins de qualité et réactifs. En Tunisie, de nombreux établissements de santé dépendent encore de méthodes manuelles et de systèmes centralisés limités, ce qui entraîne des délais dans la détection des situations critiques. Par exemple, les infirmiers doivent vérifier physiquement les constantes vitales des patients, ce qui augmente le risque d'erreurs humaines et la charge de travail, particulièrement dans les hôpitaux à ressources limitées où les équipements modernes sont rares.

Ce projet, réalisé au sein du SmartLab de la Faculté de Médecine de Monastir, s'inscrit dans une démarche d'innovation visant à moderniser la surveillance post-opératoire. Il propose un système de collecte de données en temps réel et de gestion des alertes, combinant l'extraction de données via la reconnaissance optique de caractères (OCR) pour les moniteurs. L'objectif principal est d'améliorer la réactivité du personnel médical face aux situations critiques, tout en réduisant les interventions manuelles et les erreurs.

Ce système répond à plusieurs besoins critiques dans le domaine médical :

- **Pour les patients :** Une détection rapide des anomalies vitales (fréquence cardiaque, pression artérielle, saturation en oxygène) permet une intervention immédiate, améliorant les résultats cliniques.
- **Pour le personnel médical :** Les notifications en temps réel via mobile et web réduisent la nécessité de déplacements constants, permettant aux infirmiers et médecins de se concentrer sur les cas prioritaires.
- **Pour les hôpitaux :** L'automatisation de la surveillance améliore l'efficacité opérationnelle et réduit les coûts liés aux erreurs humaines ou aux retards d'intervention.

En outre, la solution est conçue pour être évolutive, adaptable à différents types de moniteurs et conforme aux réglementations médicales en matière de sécurité des données.

Chapitre 1

Cadre Général du Projet

1.1 Introduction

Le présent chapitre vise à décrire le contexte général du projet **VitalSync**, réalisé au sein du **SmartLab** de la **Faculté de Médecine de Monastir (FMM)**. Il présente l'organisme d'accueil, le contexte hospitalier tunisien, les défis liés à la surveillance post-opératoire, ainsi que la solution proposée. Ce chapitre s'articule autour de quatre axes principaux : la présentation de l'organisme, l'analyse du contexte et des problématiques, l'étude des solutions existantes, et la description de la solution **VitalSync**.

1.2 Présentation de l'organisme d'accueil

Le projet a été réalisé au sein du **SmartLab**, une **Unité de Service Commun et de Recherche (USCR)** rattachée à la **Faculté de Médecine de Monastir (FMM)**, fondée le 15 août 1980 [1]. La FMM, institution tunisienne de référence, est reconnue pour son excellence académique et son engagement dans la recherche médicale. Certifiée **ISO 21001**, elle forme des professionnels de santé et contribue à l'avancement des sciences médicales via des projets innovants [2].

Le **SmartLab** catalyse l'innovation en santé en favorisant la collaboration entre chercheurs, cliniciens et start-ups [3]. Doté d'équipements de pointe (laboratoires IoT, outils de simulation) et d'une équipe pluridisciplinaire, il développe des solutions technologiques adaptées aux défis locaux, comme les contraintes budgétaires et les besoins en automatisation. Cette unité constitue une plateforme idéale pour moderniser la surveillance post-opératoire, en comblant le fossé entre recherche académique et applications cliniques.



FIGURE 1.1 : Logo de la FMM



FIGURE 1.2 : Logo du SmartLab

1.3 Contexte et problématique

Dans les hôpitaux tunisiens, la surveillance en salle de réveil repose sur des dispositifs classiques : un écran centralisé affiche les signes vitaux, tandis que le personnel effectue des rondes régulières pour consulter les moniteurs au chevet des patients [4]. Ce modèle, dépendant fortement de l'intervention humaine pour surveiller les constantes vitales.

1.3.1 Variabilité des signes vitaux

Les constantes vitales, telles que la fréquence cardiaque, la pression artérielle et la saturation en oxygène (SpO_2), sont essentielles pour évaluer l'état de santé des patients. Elles varient en fonction de l'âge, de l'état clinique et du contexte post-opératoire [3]. Par exemple, la fréquence cardiaque normale pour un adulte au repos est de 60 à 100 battements par minute (bpm), tandis que pour un enfant de 3 à 12 ans, elle se situe entre 70 et 120 bpm [5].

TABLE 1.1 : Plages normales des signes vitaux selon l'âge et le contexte

Signe vital	Adulte (repos)	Enfant (3–12 ans)	Post-opératoire (adulte)
Fréquence cardiaque (bpm)	60–100	70–120	70–120
Pression artérielle (mmHg)	90/60–120/80	80/50–110/70	100/60–140/90
SpO_2 (%)	95–100	95–100	92–100

1.3.2 Défis structurels et opérationnels

La méthode traditionnelle de surveillance post-opératoire adoptée par les hôpitaux tunisiens présente plusieurs limitations qui compromettent la sécurité des patients et l'efficacité des soins. Parmi les principaux défis, on note :

- **Sous-effectif** : Le ratio infirmier-patient (1 :12 contre 1 :6 recommandé par l'OMS) augmente la charge de travail et retarde la détection des anomalies [5].
- **Absence d'automatisation** : Les vérifications manuelles sont chronophages et sujettes à des erreurs, surtout lors des pics d'activité [6].
- **Hétérogénéité des équipements** : Les moniteurs varient (analogiques sans connectivité comme Philips, ou numériques avec API comme Mindray), compliquant l'intégration des données [7].

Ces contraintes soulignent le besoin d'un système économique, facilement déployable, compatible avec divers équipements, et capable d'automatiser la surveillance avec des alertes en temps réel.

1.4 Analyse des solutions existantes

Cette section est restructurée pour présenter clairement les solutions existantes, leurs limites, et la nécessité d'une solution adaptée au contexte tunisien.

1.4.1 Présentation des solutions existantes

Plusieurs solutions technologiques sont actuellement déployées à l'échelle internationale pour assurer la surveillance des paramètres vitaux en milieu hospitalier. Parmi les plus connues figurent les systèmes **Mindray**, **ChartX** et **IQ Messenger SmartApp**, chacun offrant des fonctionnalités adaptées à différents contextes cliniques.

- **Mindray** propose une solution de surveillance centralisée qui permet d'afficher en temps réel les constantes vitales de plusieurs patients sur un écran dédié, facilitant ainsi le suivi global en salle de réveil.



FIGURE 1.3 : Logo de Mindray

- **IQ Messenger SmartApp** se distingue par sa capacité à générer des alertes instantanées et par son intégration avec les systèmes d'information hospitaliers, notamment les dossiers hospitaliers électroniques (DHE), permettant une coordination optimale des soins.



FIGURE 1.4 : Logo de IQ Messenger

- **ChartX** est un système interopérable conçu pour s'intégrer à divers équipements médicaux, tout en offrant des tableaux de bord interactifs et une visualisation continue des données patient.



FIGURE 1.5 : Logo de ChartX

1.4.2 Comparaison et limites

Le Tableau 1.2 compare ces solutions selon cinq critères : automatisation, compatibilité, alertes, simplicité matérielle, et efficacité opérationnelle.

TABLE 1.2 : Comparaison des solutions de surveillance

Critère	Mindray	ChartX	IQ Messenger
Automatisation avancée	∅	±	✓
Compatibilité appareils	∅	✓	±
Alertes en temps réel	∅	±	✓
Simplicité matérielle	±	∅	✓
Efficacité opérationnelle	±	±	✓

Légende : ✓ : Optimal, ± : Moyen, ∅ : Non adapté

Analyse critique :

Plusieurs systèmes internationaux ont été développés pour améliorer la surveillance post-opératoire, parmi lesquels **Mindray**, **ChartX** et **IQ Messenger SmartApp**. Bien que ces solutions présentent des fonctionnalités avancées, leur déploiement dans le contexte tunisien reste limité pour plusieurs raisons.

- **Mindray** : Bien qu'il soit largement utilisé, ce système reste centré sur l'affichage des paramètres sur un écran centralisé, sans génération d'alertes mobiles ni mécanismes d'automatisation des réponses. De plus, il est étroitement lié à ses propres équipements, rendant son intégration avec des moniteurs d'autres marques difficile [8].
- **ChartX** : Cette plateforme repose sur une infrastructure réseau stable et performante pour fonctionner de manière optimale. Or, la connectivité réseau dans certains établissements tunisiens est insuffisante, ce qui peut compromettre son efficacité.
- **IQ Messenger SmartApp** : Elle offre une bonne réactivité grâce à la gestion des alertes et à l'envoi de notifications en temps réel. Cependant, sa compatibilité avec

les dispositifs médicaux est restreinte, et son coût élevé constitue un obstacle majeur pour les hôpitaux à budget limité.

En résumé, ces solutions ne répondent que partiellement aux besoins des établissements tunisiens. Leur coût, leur dépendance à des infrastructures techniques avancées, et leur faible compatibilité avec les équipements hétérogènes limitent fortement leur adoption. Cela renforce la nécessité de développer une solution locale, flexible et abordable, adaptée aux réalités techniques et humaines du secteur hospitalier tunisien.

Ces limitations justifient le développement d'une solution adaptée au contexte tunisien.

1.5 Solution proposée : VitalSync

Le projet **VitalSync** vise à moderniser la surveillance post-opératoire au sein des établissements de santé tunisiens en proposant une solution intelligente, économique et facilement intégrable. En combinant des technologies éprouvées telles que l'**Internet des Objets (IoT)** [9], la **reconnaissance optique de caractères (OCR)** [10] et les **notifications instantanées** via *Firebase Cloud Messaging* [11], VitalSync automatise la collecte des paramètres vitaux et assure une réactivité optimale grâce à un système d'alerte en temps réel.

1.5.1 Fonctionnement de VitalSync

Conçu pour s'adapter aux contraintes des hôpitaux tunisiens, VitalSync repose sur une architecture légère et peu coûteuse. Le système utilise des caméras IP, connectées à un serveur local, pour capturer en temps réel les informations affichées sur les écrans des moniteurs médicaux. Ces données, telles que la fréquence cardiaque, la pression artérielle ou la saturation en oxygène (SpO_2), sont ensuite extraites automatiquement grâce à une combinaison des bibliothèques *OpenCV* et *Tesseract*, via un processus de reconnaissance optique de caractères (OCR).

Les données ainsi acquises sont ensuite nettoyées, structurées et analysées par un moteur de traitement intelligent qui les compare à des seuils critiques personnalisables selon le profil du patient (âge, antécédents médicaux, type d'intervention). En cas de détection d'une valeur anormale, une alerte est générée et transmise instantanément au personnel médical sous forme de notification, via le service *Firebase Cloud Messaging*. Ce mécanisme garantit une prise de décision rapide et ciblée en cas d'urgence.

Le fonctionnement de VitalSync s'articule autour de trois grandes phases fonctionnelles, résumées dans le Tableau 1.3.

TABLE 1.3 : Domaines couverts par VitalSync

Phase	Description
Acquisition	Capture des données vitales à partir des écrans des moniteurs via OCR (OpenCV + Tesseract).
Traitement	Nettoyage, structuration, et analyse des données en temps réel pour détecter les écarts critiques selon des seuils paramétrables.
Visualisation	Affichage des données sur une interface intuitive avec tableaux de bord dynamiques et accès à l'historique pour le suivi des patients.

1.5.2 Les Avantages de VitalSync

La solution VitalSync présente de nombreux avantages par rapport aux systèmes existants. Elle est conçue pour être facilement déployable dans les hôpitaux tunisiens, sans nécessiter d'infrastructure complexe ou coûteuse. En utilisant des caméras IP standard et un serveur local, elle s'intègre aisément aux équipements déjà en place, qu'ils soient anciens ou récents. De plus, son architecture modulaire permet une évolutivité et une adaptabilité aux besoins spécifiques de chaque établissement.

VitalSync permet également de réduire considérablement la charge de travail du personnel soignant en automatisant les tâches de surveillance, tout en augmentant la réactivité face aux situations critiques.

Elle contribue ainsi à l'amélioration de la sécurité des patients et à l'optimisation de la qualité des soins. Enfin, cette approche ouvre la voie à des applications futures dans le domaine de la santé connectée en Tunisie, en posant les bases d'un écosystème technologique local durable [3].

1.6 Conclusion

Ce chapitre a permis de présenter le projet **VitalSync** en exposant le contexte dans lequel il s'inscrit, les contraintes du système de santé tunisien, ainsi que les limites des solutions existantes. Face à ces défis, VitalSync propose une réponse technologique innovante, accessible et parfaitement adaptée aux réalités locales. Les chapitres suivants détailleront la conception, le développement technique et l'évaluation de cette solution.

Chapitre 2

Méthodologie et spécifications des besoins

2.1 Introduction

Dans ce chapitre, nous présentons la méthodologie adoptée pour le développement du projet **VitalSync**, ainsi que les spécifications des besoins fonctionnels et non fonctionnels.

2.2 Analyse et spécification des besoins

L'analyse des besoins est une étape cruciale pour définir les fonctionnalités et les contraintes du système. Elle permet de s'assurer que le projet répond aux attentes des utilisateurs finaux.

2.2.1 Identification des acteurs

Le système interagit avec plusieurs acteurs essentiels, chacun jouant un rôle distinct dans son fonctionnement. Ces acteurs, résumés dans un tableau dédié, sont en phase avec le contexte médical et les composants architecturaux décrits dans les chapitres précédents.

TABLE 2.1 : Acteurs du système et leurs rôles

Acteur	Description
Médecin	Consulte les données des patients, reçoit les alertes critiques et valide les actions nécessaires.
Infirmier(ère)	Surveille les patients, reçoit les alertes en fonction de ses horaires et prend des mesures immédiates.
Administrateur du Système	Gère les utilisateurs, configure les seuils d'alerte et assure le bon fonctionnement du système.
Moniteur de Signes Vitaux	Source de données des signes vitaux, transmises via API ou OCR basé sur caméra.
Caméra	Capture les écrans des moniteurs pour l'extraction des données via YOLO et OCR.

2.2.2 Specification des besoins

La spécification des besoins est essentielle pour définir les fonctionnalités et les contraintes du système. Elle se divise en deux catégories principales : les besoins fonctionnels et les besoins non fonctionnels.

Besoins fonctionnels :

Le système **VitalSync** a pour principal objectif d'automatiser la collecte des signes vitaux, d'optimiser la gestion des alertes médicales, et de fournir une interface de suivi efficace. Les besoins fonctionnels traduisent ces objectifs sous forme de fonctionnalités concrètes :

- **Collecte automatique des données** : Le système capte en temps réel les informations affichées sur les moniteurs de signes vitaux (fréquence cardiaque, pression artérielle, SpO₂) à l'aide de caméras IP, combinées à une technologie d'OCR (OpenCV + Tesseract).
- **Gestion des alertes** : Lorsqu'un dépassement de seuil est détecté, le système génère automatiquement une alerte critique. Ces alertes peuvent être visualisées sur une interface dédiée et transmises en temps réel via des notifications (e.g., push mobile).
- **Visualisation des données** : Les utilisateurs accèdent aux constantes vitales et aux alertes via des interfaces web et mobiles. Ces interfaces offrent des tableaux de bord interactifs avec des vues temps réel et un historique structuré.
- **Gestion des utilisateurs** : Un module d'authentification et de gestion des rôles permet de restreindre l'accès selon le profil utilisateur (médecins, infirmiers, administrateurs).
- **Archivage des données** : Toutes les mesures collectées et alertes générées sont stockées dans une base de données relationnelle, assurant un historique exploitable pour le suivi médical et les audits.

Besoins non fonctionnels :

Les besoins non fonctionnels garantissent la qualité, la robustesse et la conformité du système dans un environnement hospitalier sensible. Ils s'articulent autour des axes suivants :

- **Disponibilité** : Le système doit fonctionner de manière continue, avec un temps d'arrêt minimal, afin d'assurer un suivi constant des patients.
- **Sécurité des données** : Toutes les données sensibles relatives aux patients doivent être protégées par des mécanismes de chiffrement robustes et une authentification sécurisée. Le système respecte les normes de confidentialité en vigueur.
- **compatibilité** : VitalSync doit être compatible avec une grande diversité de moniteurs médicaux, qu'ils soient anciens ou récents, issus de différents fabricants.

2.3 Choix de la méthodologie

Dans le cadre du développement d'un système de collecte de données en temps réel et de gestion des alertes dans un environnement médical, il existe plusieurs méthodologies de développement logiciel, chacune ayant ses avantages et inconvénients. Parmi les méthodologies les plus courantes, on trouve Scrum, Waterfall, Lean et Kanban.

2.3.1 Etude comparative des méthodologies

Le tableau suivant (Tableau 2.2) présente une comparaison des principales méthodologies de développement logiciel mentionnées, en mettant en évidence les caractéristiques clés de chaque méthodologie, telles que l'adaptabilité aux changements, les livraisons fréquentes, la collaboration renforcée et la gestion des risques.

TABLE 2.2 : tableau comparatif des méthodologies de développement logiciel.

Caractéristiques	Scrum	Waterfall	Lean	Kanban
Adaptabilité aux changements	✓	X	X	✓
Livraisons fréquentes	✓	X	X	X
Collaboration renforcée	✓	X	X	✓
Réponse rapide aux changements	✓	X	X	X
Gestion des risques	✓	✓	✓	✓
Niveau d'implication du client	Élevé	Faible	Moyen	Moyen
Efficacité des coûts	Élevée	Moyenne	Élevée	Moyenne
Assurance qualité	Continue	Finale	Continue	Continue
Scalabilité	Moyenne	Élevée	Élevée	Moyenne
Maintenance et support	Facile	Difficile	Facile	Facile

2.3.2 Méthodologie retenue : Scrum

A l'issue de la comparaison des différentes approches, la méthodologie Scrum a été retenue. Cette décision est basée sur plusieurs critères clés qui correspondent aux besoins spécifiques du développement d'un système de collecte de données en temps réel et de gestion des alertes dans un environnement médical.

Premièrement, l'approche itérative de Scrum permet de recueillir des retours continus des parties prenantes, notamment le personnel médical du SmartLab de la Faculté de Médecine de Monastir, pour affiner les exigences fonctionnelles et non fonctionnelles.

Cela est particulièrement crucial dans un projet médical où la précision et la réactivité sont essentielles pour garantir la sécurité des patients.

Deuxièmement, Scrum offre une grande adaptabilité face aux incertitudes techniques, telles que l'extraction de données à partir de moniteurs via OCR. Ces sources de données présentent des défis distincts, notamment en termes de précision de l'OCR. Scrum permet de tester et valider ces composants dès les premiers sprints, réduisant ainsi les risques d'échec technique.

Enfin, la méthodologie Scrum favorise une collaboration étroite entre les développeurs, le Product Owner et le Scrum Master, assurant une communication fluide avec les parties prenantes médicales. Cette collaboration est essentielle pour répondre aux exigences changeantes du domaine médical, où les besoins peuvent évoluer rapidement en fonction des retours des utilisateurs ou des contraintes réglementaires [12, 13].

2.4 Présentation de Scrum

Scrum est un framework agile qui repose sur des cycles de développement appelés sprints, comme présenté dans la figure 2.1. Chaque sprint a une durée définie (généralement entre 2 et 4 semaines) et aboutit à un produit fonctionnel et potentiellement livrable. Le processus Scrum comprend plusieurs éléments clés tels que le *Product Backlog*, le *Sprint Backlog*, l'*Incrément*, ainsi que des événements tels que la planification de sprint, les réunions quotidiennes (*Daily Scrum*), la revue de sprint et la rétrospective de sprint [14].

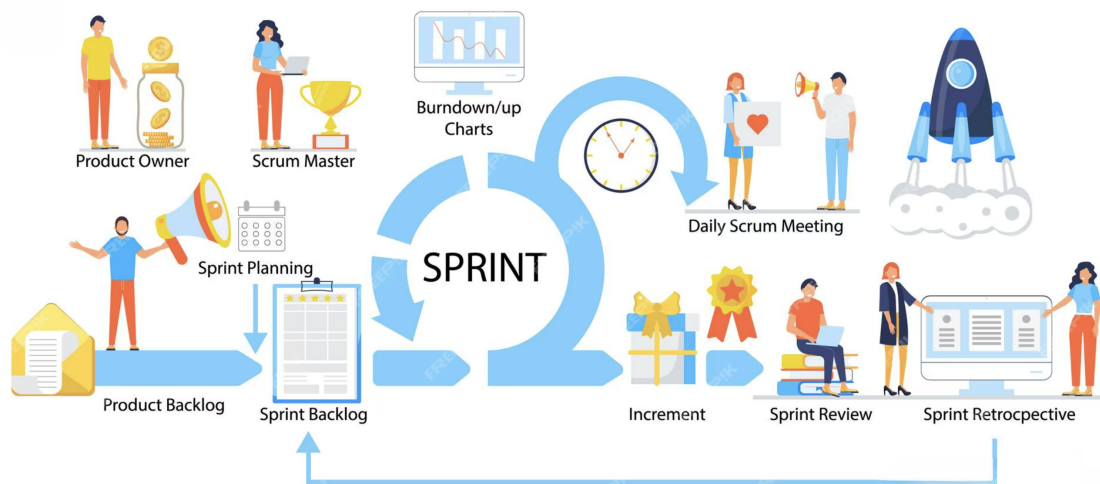


FIGURE 2.1 : Processus Scrum.

2.4.1 Artefacts Scrum et leur application

Scrum repose sur trois artefacts principaux qui structurent le développement du projet : le *Product Backlog*, le *Sprint Backlog* et l'*Incrément*. Ces artefacts sont appliqués comme suit dans le cadre du projet [15] :

- **Product Backlog** : Le *Product Backlog* regroupe l'ensemble des fonctionnalités à développer, telles que définies dans le tableau 4.2 (voir section 4.2.2). Il est géré par le Product Owner, qui priorise les tâches en utilisant la méthode MoSCoW pour classer les fonctionnalités en *Must*, *Should*, *Could* et *Won't*. Ce backlog est régulièrement affiné pour intégrer les retours des parties prenantes, notamment le personnel médical, afin de garantir que les fonctionnalités prioritaires répondent aux besoins critiques du système.
- **Sprint Backlog** : À chaque sprint, un sous-ensemble de tâches est sélectionné à partir du *Product Backlog* pour constituer le *Sprint Backlog*. Par exemple, pour le Sprint 1, les tâches se concentrent sur l'intégration de la collecte de données basée sur caméra et l'extraction OCR (voir section ??). Ces tâches sont définies lors de la réunion de planification du sprint, en collaboration avec l'équipe de développement.
- **Incrément** : Chaque sprint produit un incrément fonctionnel et potentiellement livrable. Par exemple, à l'issue du Sprint 1, un pipeline OCR fonctionnel est livré, permettant la capture et l'extraction des données vitales à partir des moniteurs. Cet incrément peut être testé par le personnel médical pour valider sa précision et sa fiabilité avant d'intégrer de nouvelles fonctionnalités dans les sprints suivants.

2.4.2 Événements Scrum et leur implémentation

Scrum repose sur des événements clés qui structurent le processus de développement. Ces événements sont implémentés comme suit dans le cadre du projet [12] :

- **Planification de sprint** : Lors de la réunion de planification, l'équipe sélectionne les tâches du *Product Backlog* à inclure dans le *Sprint Backlog*, en estimant l'effort requis (voir tableau ?? pour le Sprint 2). Par exemple, pour le Sprint 1, l'intégration de l'OCR a été estimée à 5 jours, en tenant compte de la configuration des caméras et du pipeline de traitement d'images.
- **Daily Scrum** : Des réunions quotidiennes de 15 minutes sont organisées pour suivre l'avancement des tâches et identifier les obstacles. Par exemple, si des problèmes de qualité d'image affectent l'OCR, ils sont discutés et résolus rapidement, souvent en ajustant les paramètres de prétraitement d'OpenCV.

- **Revue de sprint** : À la fin de chaque sprint, l'incrément est présenté aux parties prenantes, comme le pipeline OCR fonctionnel à la fin du Sprint 1. Les médecins et infirmiers testent l'incrément pour valider sa précision et son utilité clinique, fournissant des retours pour le sprint suivant.
- **Rétrospective de sprint** : Après chaque sprint, l'équipe analyse les succès et les défis rencontrés. Par exemple, si des erreurs d'OCR sont détectées lors du Sprint 1, la rétrospective peut conduire à l'ajout de tests supplémentaires ou à l'amélioration du prétraitement des images dans le Sprint 2.

2.5 Les rôles Scrum

Dans Scrum, trois rôles principaux sont définies pour assurer le bon fonctionnement du processus de développement [16] :

- **Product Owner** : Représentant des parties prenantes, il définit et priorise les fonctionnalités à développer en fonction des besoins des utilisateurs et des objectifs du projet.
- **Scrum Master** : Responsable de la bonne application du framework Scrum. Il facilite la communication, aide à surmonter les obstacles et veille à l'amélioration continue des processus.
- **Équipe de développement** : Composée de développeurs, designers et autres spécialistes, elle est responsable de la mise en œuvre des fonctionnalités et de la livraison des incréments du produit.

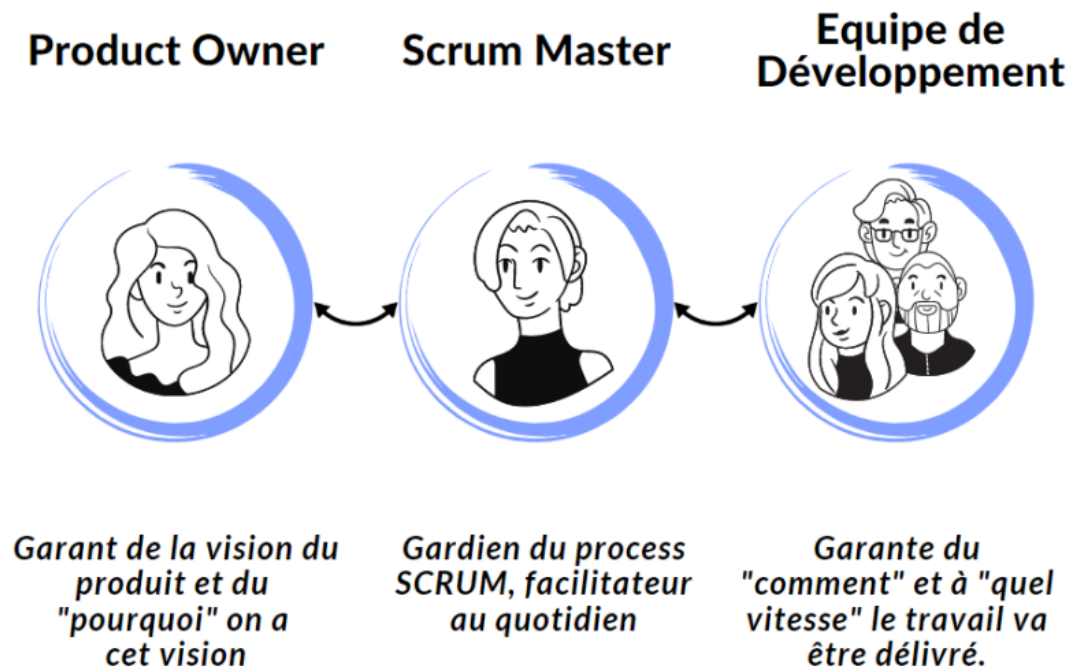


FIGURE 2.2 : Rôles Scrum.

2.6 Outils de support à Scrum

Pour faciliter la mise en œuvre de Scrum, plusieurs outils numériques sont utilisés pour gérer les processus, la communication et le suivi des tâches :

- **ClickUp** : Utilisé pour gérer le *Product Backlog* et le *Sprint Backlog*, ClickUp permet de suivre les tâches, d'assigner des responsabilités et de visualiser l'avancement des sprints à l'aide de tableaux Kanban ou Scrum [17]. Par exemple, les tâches du Sprint 1, comme l'intégration de l'OCR (M-02), sont suivies via des tickets ClickUp avec des estimations de temps et des statuts mis à jour quotidiennement.



FIGURE 2.3 : Logo de ClickUp.

- **Slack** : Cette plateforme de communication permet des échanges en temps réel entre l'équipe de développement, le Product Owner et le Scrum Master [18]. Les *Daily Scrum* peuvent être partiellement gérés via des canaux dédiés, où les membres signalent leurs progrès et obstacles.



FIGURE 2.4 : Logo de Slack.

- **Git :** Utilisé pour le contrôle de version, Git permet de gérer le code source du projet et de faciliter la collaboration entre les développeurs [19]. Les commits sont alignés avec les tâches du *Sprint Backlog*, garantissant une traçabilité des contributions.



FIGURE 2.5 : Logo de Git.

2.7 Conclusion

Ce chapitre a présenté la méthodologie Scrum adoptée pour le projet **VitalSync**, en détaillant les spécifications des besoins fonctionnels et non fonctionnels, ainsi que les rôles, artefacts et événements Scrum.

Chapitre 3

Étude architecturale du projet

3.1 Introduction

Ce chapitre décrit l'architecture du système VitalSync. L'objectif est d'expliquer les choix technologiques, les parties du système et leurs interactions pour répondre aux besoins identifiés. L'architecture est simple, adaptable et facile à maintenir.

3.2 Présentation de l'architecture VitalSync

VitalSync utilise une architecture en trois parties : collecte de données, traitement et stockage, et affichage et alertes. Chaque partie a un rôle clair, comme montré dans la Figure 3.1. Cette organisation permet de gérer les données médicales efficacement et d'envoyer des alertes rapidement.

- **Collecte de données** : Cette partie récupère les informations des moniteurs médicaux à l'aide de caméras IP. Les caméras capturent les écrans des moniteurs qui affichent des données comme le rythme cardiaque ou l'oxygène dans le sang (SpO2).
- **Traitement et stockage** : Cette partie traite les images capturées et sauvegarde les données. Les images sont améliorées avec OpenCV, les zones importantes sont détectées avec YOLO, et les valeurs sont extraites avec Tesseract OCR. Un service Flask gère les images, et Laravel organise les utilisateurs, les données et les alertes. Une base MySQL stocke tout.
- **Affichage et alertes** : Cette partie montre les données aux utilisateurs via une application web React et une application mobile Flutter. Les alertes sont envoyées en temps réel avec Firebase Cloud Messaging (FCM) ou par e-mail pour les cas urgents.

Architecture du système VitalSync

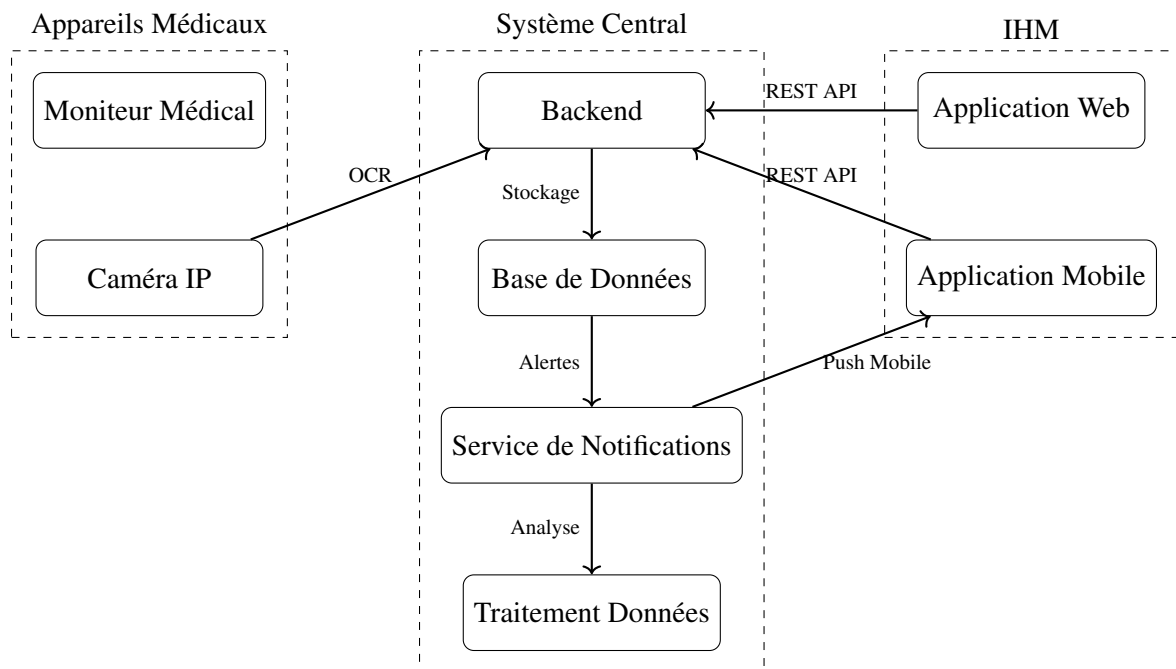


FIGURE 3.1 : Architecture générale du système VitalSync

3.3 Modèle de conception logicielle adoptée

Pour organiser le système VitalSync, nous avons choisi le modèle MVC (Modèle-Vue-Contrôleur). Ce modèle est adapté pour séparer les données, l’affichage et la logique de traitement, ce qui facilite la maintenance et les tests. Voici comment nous l’avons appliqué :

- **Modèle** : Cette partie gère les données. La base MySQL stocke les informations des patients, des mesures et des alertes. Laravel traite les requêtes et les interactions avec la base de données.
- **Vue** : Cette partie montre les données aux utilisateurs. Les applications React (web) et Flutter (mobile) affichent des tableaux et des alertes de manière simple.
- **Contrôleur** : Cette partie gère les actions. Laravel et Flask contrôlent le traitement des images, l’envoi des alertes et les interactions des utilisateurs.

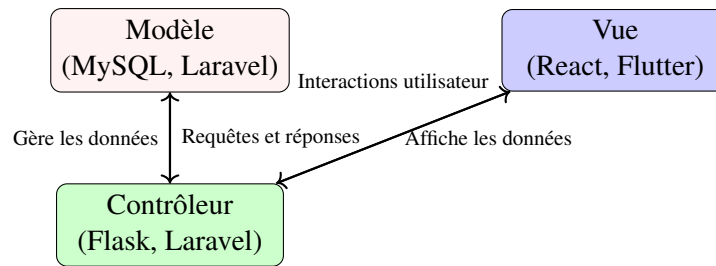


FIGURE 3.2 : Modèle MVC de VitalSync

Cette structure permet de séparer clairement les responsabilités, rendant le système plus modulaire et facile à étendre. Par exemple, si nous voulons ajouter une nouvelle fonctionnalité d'alerte, nous pouvons le faire dans le contrôleur sans toucher à la vue ou au modèle.

3.4 Interactions et fonctionnement du système

Cette section présente les diagrammes UML qui montrent comment les parties du système travaillent ensemble.

3.4.1 Diagramme des cas d'utilisation

Le diagramme des cas d'utilisation (Figure 3.3) montre comment les utilisateurs (médecins, infirmiers, administrateurs) et les appareils (moniteurs, caméras) interagissent avec le système. Il illustre les actions comme consulter les données ou recevoir des alertes.

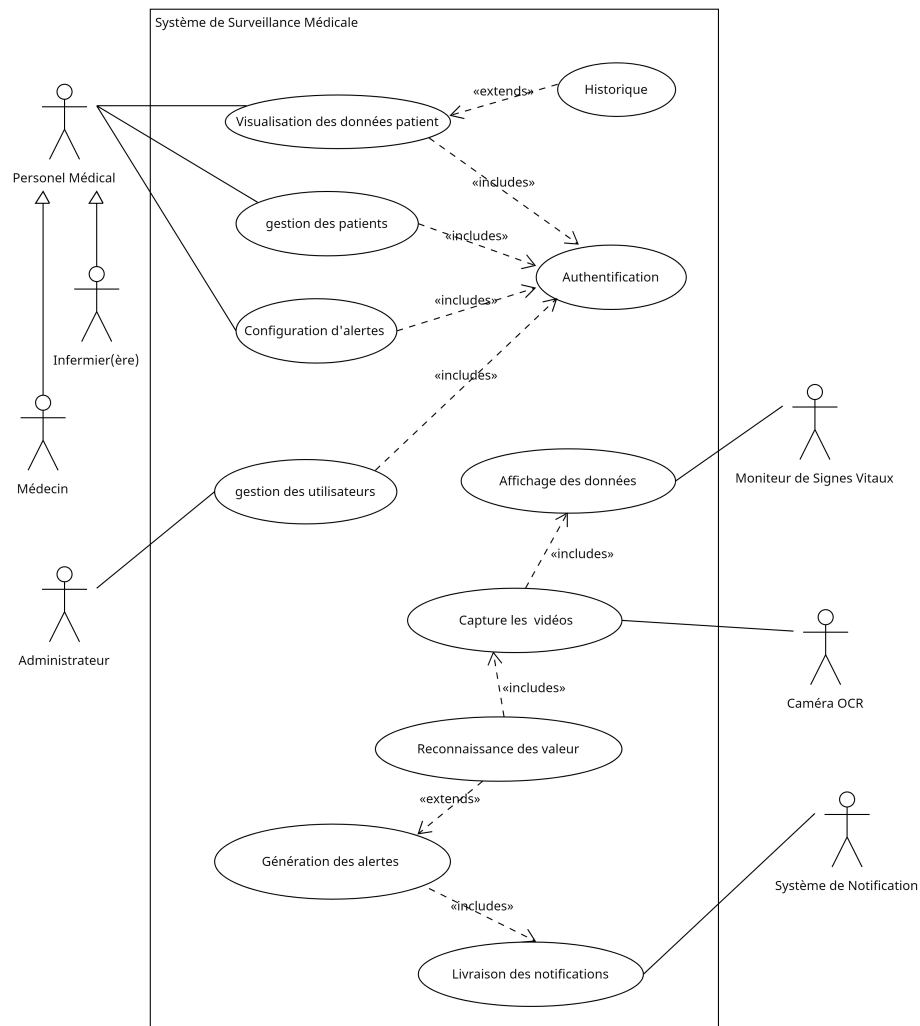


FIGURE 3.3 : Diagramme des cas d'utilisation global

3.4.2 Diagramme de classes

Le diagramme de classes (Figure 3.4) montre la structure de la base de données, avec des entités comme Patient, Mesure et Alerte. Il explique comment les données sont organisées. Voir le Chapitre 5 pour plus de détails.

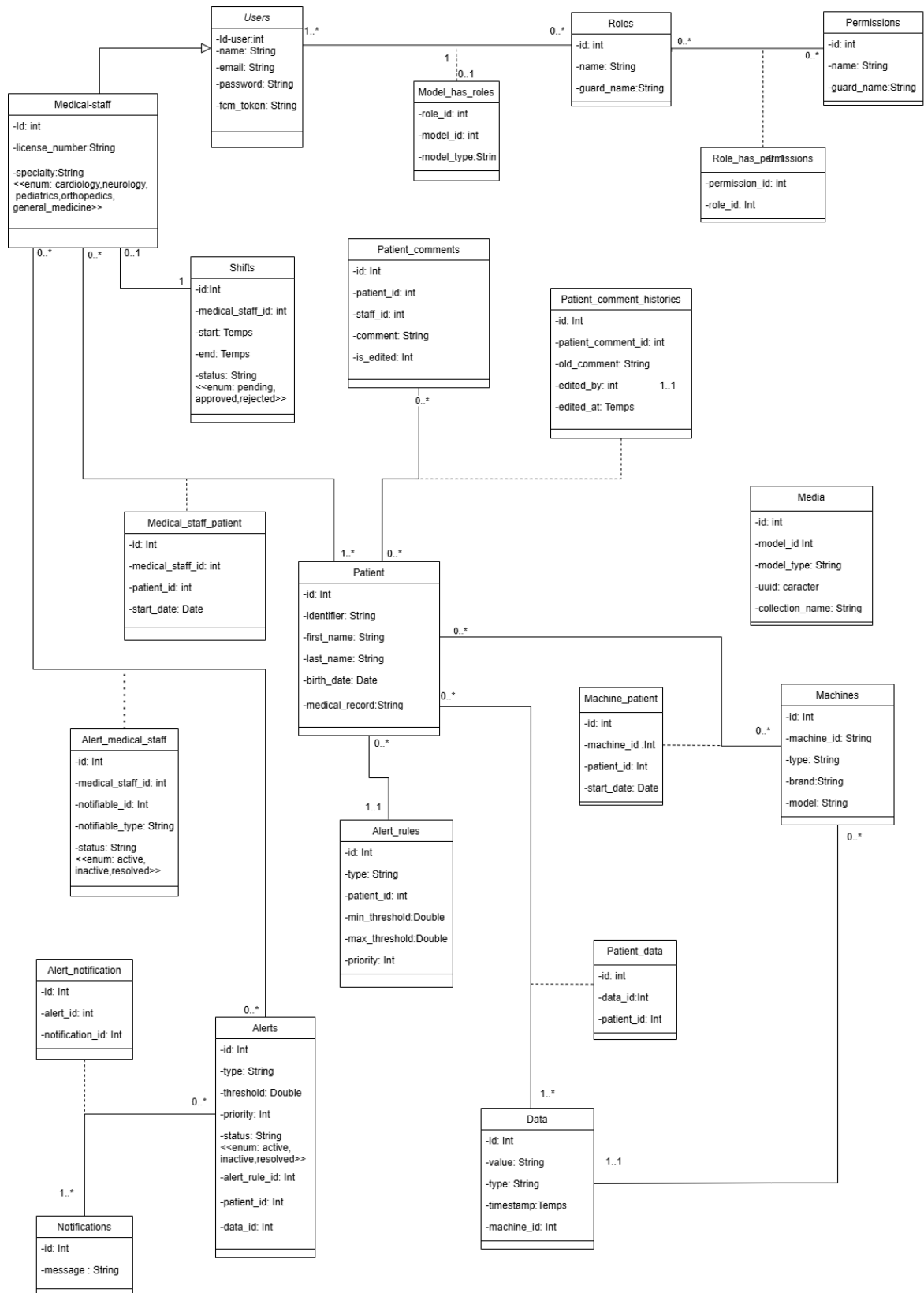


FIGURE 3.4 : Diagramme de classes

3.4.3 Diagramme de séquence

Le diagramme de séquence (Figure 3.5) décrit le processus depuis la capture des images par les caméras jusqu'à l'envoi des alertes via FCM. Il montre les étapes du traitement des données.

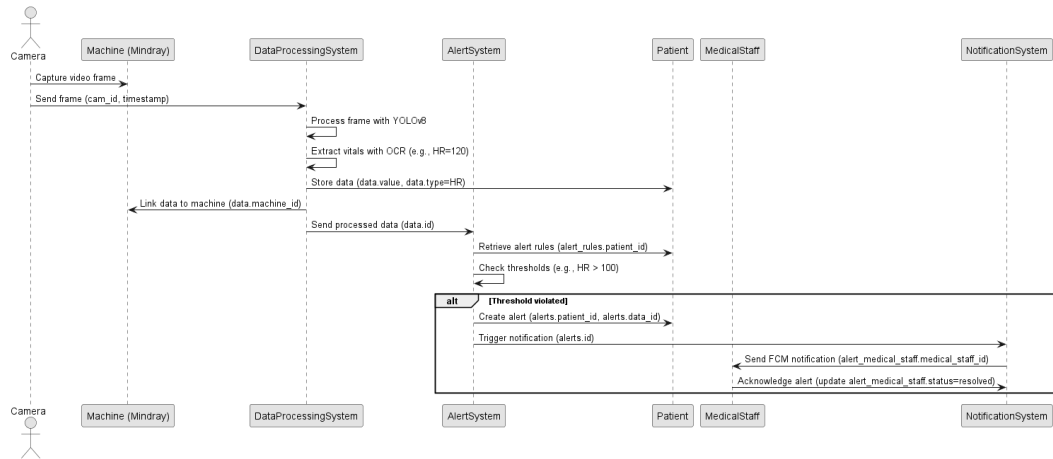


FIGURE 3.5 : Diagramme de séquence global

3.4.4 Diagramme d'activités

Le diagramme d'activités (Figure 3.6) montre les étapes du traitement des données, de la collecte à l'envoi des alertes. Il explique comment les anomalies sont détectées et signalées.

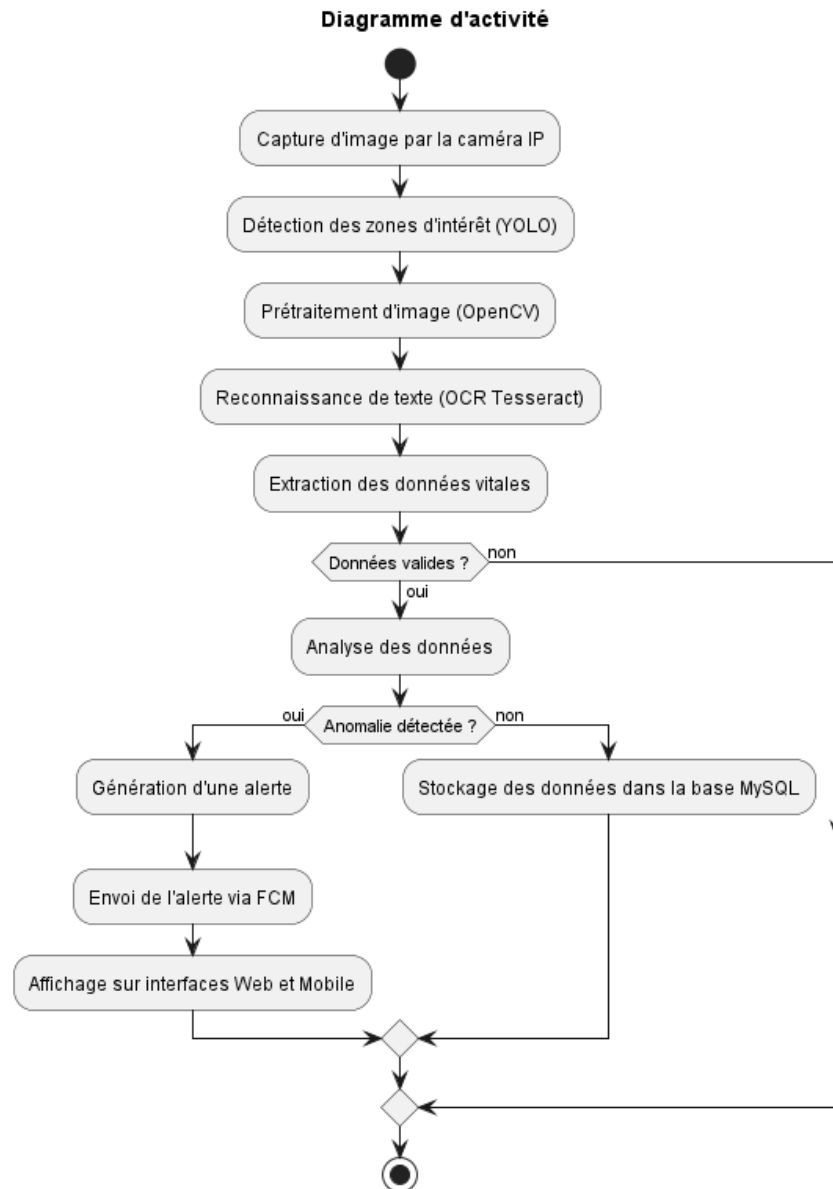


FIGURE 3.6 : Diagramme d'activité du flux de données

3.4.5 Diagramme de composants

Le diagramme de composants (Figure 3.7) montre les modules logiciels (Flask, Laravel, React, Flutter) et comment ils sont connectés.

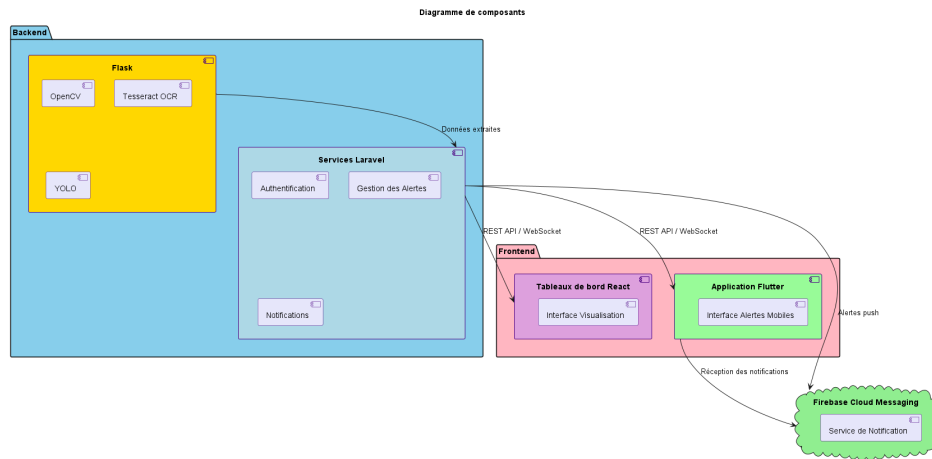


FIGURE 3.7 : Diagramme de composants du système

3.4.6 Diagramme de déploiement

Le diagramme de déploiement (Figure 3.8) montre l'organisation physique du système, avec les serveurs, caméras et appareils mobiles.

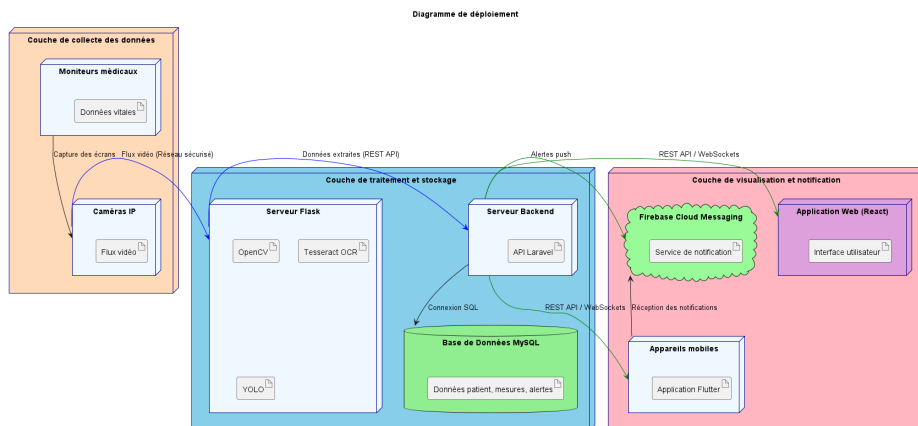


FIGURE 3.8 : Diagramme de déploiement

3.5 Pourquoi ce choix d'architecture

Cette architecture est choisie pour plusieurs raisons :

- **Simplicité** : Les trois parties (collecte, traitement, affichage) sont séparées, ce qui facilite les modifications et la maintenance.
- **Rapidité** : Les alertes sont envoyées en temps réel grâce à YOLO pour la détection et FCM pour les notifications.

- **Adaptabilité** : Le système fonctionne avec différents moniteurs médicaux et peut être déployé sur des serveurs locaux ou dans le cloud.
- **Sécurité** : Les données sont protégées par HTTPS et Sanctum, garantissant la confidentialité des informations médicales.

Ces choix répondent aux besoins des hôpitaux tunisiens et respectent les principes du master Génie Logiciel, en assurant une architecture claire, adaptable et sécurisée.

3.6 Conclusion

Ce chapitre a décrit l'architecture de VitalSync, qui permet de collecter, traiter et afficher des données médicales en temps réel. L'organisation en trois parties et l'utilisation du modèle MVC rendent le système clair, adaptable et sécurisé. Cette architecture répond aux besoins des hôpitaux et prépare le système pour les prochaines étapes, comme expliqué dans les chapitres suivants.

Chapitre 4

Sprint 0 : Planification et préparation du projet

4.1 Introduction

Le Sprint 0 constitue la phase initiale du projet, qui vise à développer un système de collecte de données en temps réel et de gestion des alertes pour les appareils médicaux en salle de réveil. Cette étape fondamentale permet d'identifier les acteurs clés, de recueillir les besoins fonctionnels et non fonctionnels, et de définir la méthodologie Scrum qui guidera les phases ultérieures du développement. L'objectif est d'assurer un alignement avec les ambitions du projet, à savoir améliorer la surveillance post-opératoire grâce à une collecte automatisée des données et à des alertes en temps réel, comme décrit dans les chapitres d'introduction générale et d'étude architecturale.

4.2 Pilotage du projet avec Scrum

Le projet adopte la méthodologie Scrum pour assurer un développement itératif et une collaboration étroite avec les parties prenantes, comme justifié dans le chapitre consacré à la méthodologie. Cette section détaille les rôles Scrum, l'organisation du backlog et la planification des sprints.

4.2.1 Acteurs Scrum et Missions

Les rôles Scrum et leurs responsabilités sont définis dans un tableau spécifique, garantissant une approche collaborative alignée sur les principes décrits précédemment. Le Product Owner est chargé de définir et de prioriser les fonctionnalités du produit, tout en validant les livrables pour s'assurer qu'ils répondent aux besoins du projet. L'équipe de développement conçoit, développe et teste les fonctionnalités, en veillant à la qualité du code produit. Le Scrum Master facilite les événements Scrum, élimine les obstacles et veille à ce que l'équipe adhère aux pratiques de la méthodologie.

TABLE 4.1 : Acteurs du projet Scrum

Rôle	Mission	Acteur
Product Owner	Définir et prioriser les fonctionnalités du produit (Product Backlog), valider les livrables.	Pr Ammeri Charfeddine
Équipe de Développement	Concevoir, développer et tester les fonctionnalités, assurer la qualité du code.	Zaibi Racha
Scrum Master	Faciliter les événements Scrum, supprimer les obstacles, assurer l'adhésion à Scrum.	Ouni Ghada

4.2.2 Organisation du Backlog

Le Product Backlog est géré à l'aide d'un outil de gestion de projet, organisé selon la méthode de priorisation MoSCoW pour refléter les objectifs stratégiques du projet. Chaque fonctionnalité est formulée sous forme de User Story pour garantir un développement centré sur l'utilisateur. Les tâches prioritaires incluent la détection et la reconnaissance des valeurs vitales à l'aide de technologies de détection et d'OCR, la génération et la gestion des alertes basées sur des seuils prédéfinis, ainsi que l'envoi de notifications via des canaux multiples, tels que les notifications push et les e-mails. Les interfaces web et mobiles permettent la visualisation des données et la gestion des alertes, tandis que l'authentification et la gestion des utilisateurs assurent un accès sécurisé basé sur les rôles. Le stockage des données et des historiques dans une base de données relationnelle est également prioritaire, tout comme la conformité aux réglementations médicales en matière de sécurité des données. D'autres tâches, jugées importantes mais non critiques pour la première version, incluent la configuration personnalisée des seuils d'alerte par patient, la gestion des horaires pour le filtrage des alertes, et l'optimisation des notifications pour une faible latence. Des fonctionnalités optionnelles, comme le support multilingue ou la gestion des alertes par niveau de priorité, pourraient être ajoutées si les ressources le permettent. Enfin, certaines tâches, telles que l'analyse avancée des données avec intelligence artificielle, sont hors du périmètre actuel.

TABLE 4.2 : Backlog du Projet avec Priorités MoSCoW

ID	Tâche	Priorité
M-01	Détection et reconnaissance des valeurs vitales via YOLO et OCR pour les moniteurs.	M
M-02	Génération et gestion des alertes en fonction des seuils prédéfinis.	M
M-03	Envoi d'alertes via notifications push et e-mail.	M
M-04	Interface web et mobile pour visualiser les données et gérer les alertes.	M
M-05	Authentification et gestion des utilisateurs (médecins, infirmiers, administrateurs).	M
M-06	Stockage et historique des données dans une base de données.	M
M-07	Conformité aux réglementations médicales (sécurité des données).	M
S-02	Configuration des seuils d'alerte personnalisés par patient.	S
S-03	Gestion des horaires de filtrage des alertes pour le personnel médical.	S
S-04	Interface d'administration pour configurer les seuils et gérer les utilisateurs.	S
S-05	Optimisation des notifications push pour faible latence.	S
C-04	Support multilingue pour l'interface utilisateur.	C
C-05	Gestion des alertes par priorité (critique, moyen, faible).	C
C-06	Historique des alertes avec filtrage par date, patient, ou type d'alerte.	C
W-03	Analyse avancée des données avec intelligence artificielle pour le diagnostic.	W

4.2.3 Planification des sprints

Le projet est structuré en trois sprints de trois semaines chacun. Le tableau ci-dessous présente une synthèse des objectifs, livrables, tâches, critères d'acceptation et risques associés à chaque sprint.

TABLE 4.3 : Planification des Sprints du Projet VitalSync

Sprint	Objectifs et livrables	Tâches principales	Critères d'acceptation	Risques et solutions
Sprint 1 Pipeline OCR	Identification des acteurs ; recueil des besoins ; définition de la méthodologie Scrum.	Dataset annoté ; modèle YOLO entraîné ; pipeline OCR fonctionnel.	Capture et annotation ; entraînement modèle ; intégration Tesseract ; configuration caméras.	Détection précise ; extraction fiable ; faible latence ; compatibilité multi-marques. Mauvais éclairage → prétraitement ; dataset limité → augmentation.
Sprint 2 Alertes temps réel	Moteur d'alerte ; notifications selon rôles ; intégration FCM.	Configuration des seuils ; mapping rôles ; messagerie ; validation données.	Alertes rapides ; notifications correctes ; données validées avec précision.	Retards → tests simulés ; erreurs → tests unitaires.
Sprint 3 Interfaces et dashboard	Dashboard web ; app mobile ; base de données des alertes.	Développement dashboard ; design app ; notifications ; mises à jour en temps réel.	Interfaces réactives ; réception fiable ; intégrité des données ; bonne ergonomie.	Latence → optimisation réseau ; intégration complexe → prototypes/tests.

4.3 Environnement de travail

L'environnement de travail pour le projet VitalSync est structuré pour répondre aux exigences techniques et fonctionnelles définies dans les chapitres précédents. Il comprend des aspects matériels, logiciels et de développement, garantissant une intégration fluide des différentes composantes du système.

4.3.1 Environnement matériel

L'environnement matériel est conçu pour répondre aux exigences architecturales décrites précédemment. Des caméras IP capturent les écrans des moniteurs médicaux pour permettre le traitement des données par détection et OCR. Des serveurs locaux dédiés assurent le stockage, le traitement des données et la génération des alertes. Un réseau local sécurisé

avec accès Internet facilite la transmission des données entre les caméras, les serveurs et les applications. Enfin, des smartphones et tablettes compatibles permettent aux utilisateurs d'accéder aux notifications et aux données en temps réel.

4.3.2 Environnement de développement

L'environnement de développement s'appuie sur un ensemble cohérent de langages de programmation, frameworks et bibliothèques, choisis en fonction des exigences techniques du projet VitalSync.

Chaque composant du système est développé avec des outils adaptés à sa fonction :

- **PHP avec Laravel** : utilisé pour développer l'API backend principale. Laravel offre une architecture MVC robuste, une gestion avancée de l'authentification (Sanctum), ainsi qu'un système de routage et de notifications efficace.



FIGURE 4.1 : Logos des technologies backend : Laravel et PHP

- **Python avec Flask** : utilisé pour les microservices de traitement d'image. Flask, léger et modulaire, est associé à des bibliothèques comme OpenCV (prétraitement d'image), YOLO (détection des zones d'intérêt) et Tesseract OCR (extraction de texte).



FIGURE 4.2 : Logos des technologies utilisées pour le traitement et l'API Python : Python et Flask

- **Dart avec Flutter** : utilisé pour l'application mobile multiplateforme. Flutter permet de développer des interfaces fluides et réactives, compatibles Android/iOS, et intègre Firebase Cloud Messaging (FCM) pour les notifications.



FIGURE 4.3 : Logos des technologies utilisées pour le développement mobile : Flutter et Dart

- **JavaScript avec React** : utilisé pour l'interface web destinée au personnel médical. React assure une expérience utilisateur moderne avec une gestion efficace de l'état des composants et des interactions dynamiques.



FIGURE 4.4 : Logos des technologies utilisées pour le développement web : React et JavaScript

- **MySQL avec phpMyAdmin :** MySQL est la base de données relationnelle choisie pour le stockage des patients, mesures vitales, et alertes. phpMyAdmin facilite la visualisation et la gestion de la base.



FIGURE 4.5 : Logos des outils de gestion de base de données : MySQL et phpMyAdmin

Ces technologies sont intégrées dans un environnement modulaire et conteneurisé à l'aide de Docker, garantissant une portabilité entre les environnements de développement, de test et de production. Le développement collaboratif est facilité par GitHub, tandis que l'édition de code est principalement réalisée avec Visual Studio Code et Android Studio.

4.3.3 Environnement logiciel

L'environnement logiciel repose sur des systèmes d'exploitation adaptés pour le développement et les appareils mobiles. Une base de données relationnelle garantit un stockage fiable et conforme aux standards. Les communications s'appuient sur des protocoles sécurisés, tels que des API REST avec authentification par tokens et des WebSockets pour les mises à jour en temps réel. Les outils de développement suivants sont employés :

- **Visual Studio Code :** Utilisé comme environnement de développement intégré, Visual Studio Code permet de coder, tester et déboguer efficacement grâce à son interface intuitive et ses extensions pour divers langages et frameworks.



FIGURE 4.6 : Logo de Visual Studio Code

- **Android Studio :** Android Studio est utilisé pour le développement et le test des applications mobiles, offrant des outils spécialisés pour la création d'interfaces utilisateur et la gestion des performances sur les appareils Android.



FIGURE 4.7 : Logo d'Android Studio

- **LaTeX** : LaTeX est utilisé pour la rédaction du rapport final, offrant une mise en forme professionnelle et structurée. Il permet de générer des documents de haute qualité avec une gestion précise des sections, des références et des figures.



FIGURE 4.8 : Logo de LaTeX

- **GitHub** : GitHub facilite la gestion des versions et la collaboration sur le code, assurant un suivi rigoureux des modifications et une intégration fluide avec d'autres outils de développement.



FIGURE 4.9 : Logo de GitHub

- **Docker Desktop** : Docker Desktop permet de conteneuriser les applications, assurant une portabilité et une cohérence des environnements logiciels à travers les différentes phases du projet.



FIGURE 4.10 : Logo de Docker Desktop

4.4 Conclusion

Le Sprint 0 pose une base solide pour le projet en définissant les acteurs, les besoins et le cadre Scrum. Le Product Backlog priorisé, la planification détaillée des sprints et l'environnement de développement spécifié garantissent un alignement avec les objectifs de collecte de données en temps réel et de gestion des alertes. L'intégration de technologies de détection et d'OCR

positionne le système pour un développement efficace dans les sprints suivants, tout en restant fidèle à l'architecture décrite dans les chapitres précédents.

Chapitre 5

Sprint 1 : Création du dataset et mise en place du pipeline OCR

5.1 Introduction

Ce chapitre décrit le Sprint 1, première itération du développement d'un système de collecte de données en temps réel et de gestion des alertes pour les appareils médicaux en salle de réveil, comme présenté dans les chapitres d'introduction générale et d'étude architecturale. Selon la planification établie dans le chapitre précédent, cette phase se concentre sur la création d'un dataset annoté à partir de moniteurs médicaux et sur le développement d'un pipeline OCR basé sur des technologies de détection et de reconnaissance de texte pour extraire les signes vitaux, tels que la fréquence cardiaque, la pression artérielle et la saturation en oxygène. Cette itération pose les bases techniques pour la collecte automatisée des données, un objectif central du projet, en suivant la méthodologie Scrum décrite dans le chapitre consacré à la méthode.

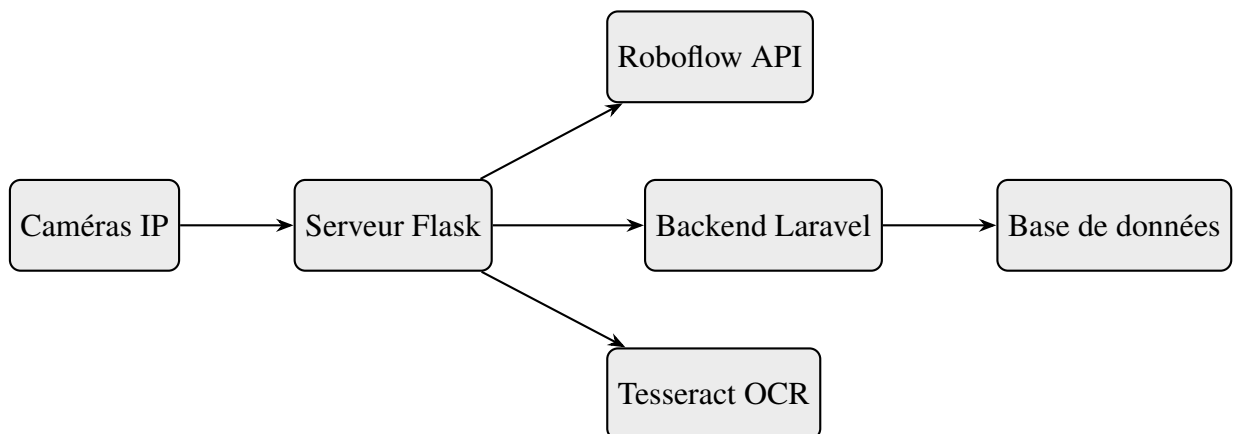


FIGURE 5.1 : Architecture du système pour le Sprint 1

5.2 Objectifs du Sprint 1

L'objectif principal du Sprint 1 est de développer un pipeline fonctionnel permettant la collecte en temps réel des données affichées sur les écrans des moniteurs à l'aide de caméras

IP. Cette phase vise à créer un dataset annoté d'images des moniteurs, avec des annotations précises pour identifier les signes vitaux. Un modèle de détection YOLO est entraîné pour repérer les régions d'intérêt contenant ces données. Des outils de traitement d'image et de reconnaissance optique de caractères (OCR) avec Tesseract sont intégrés pour extraire les valeurs textuelles. Les caméras IP sont configurées pour garantir une capture stable et optimale des images. Une API REST, implémentée avec Flask, est développée pour permettre l'intégration du pipeline dans les phases suivantes, avec une documentation Swagger et une authentification par clé API. Ces objectifs s'alignent sur les tâches prioritaires du Product Backlog et les besoins fonctionnels d'interopérabilité avec différents moniteurs, comme détaillé dans le chapitre précédent.

Sprint Backlog

Le Sprint Backlog pour le Sprint 1 regroupe les tâches sélectionnées du Product Backlog (voir Tableau 4.2) pour répondre aux objectifs de cette itération. Chaque tâche est associée à une priorité, une estimation de l'effort en heures, et des critères d'acceptation pour garantir la qualité des livrables. Les tâches incluent la configuration du matériel, la capture et l'annotation des images, l'entraînement du modèle de détection, et le développement du pipeline OCR avec une API REST.

TABLE 5.1 : Sprint Backlog

ID	Tâche	Priorité	Effort (h)	Critères d'Acceptation
M-01	Détection et reconnaissance des valeurs vitales via YOLO et OCR pour les moniteurs.	M	80	Précision de détection > 90%, OCR extrait les valeurs avec < 5% d'erreurs, compatible avec 3 moniteurs différents.
M-06	Stockage et historique des données dans une base de données.	M	20	Base de données relationnelle configurée, stockage des données extraites testé avec succès.
S-04	Interface Swagger pour l'API REST.	M	10	Documentation de l'API générée, accessible via Swagger UI, toutes les routes documentées.

5.3 Tâches réalisées

Les tâches réalisées reflètent les engagements pris lors de la planification du Sprint 1 et sont organisées en sous-sections pour plus de clarté.

5.3.1 Configuration du matériel

L'environnement matériel a été préparé pour répondre aux besoins du pipeline. Des caméras IP ont été configurées à l'aide d'une application mobile dédiée, offrant une interface intuitive pour ajuster la résolution et l'exposition. Un serveur local, équipé d'un système d'exploitation Linux avec accélération GPU, a été mis en place pour supporter l'entraînement du modèle YOLO et l'exécution du pipeline OCR. Les bibliothèques nécessaires, telles que OpenCV, Tesseract, et les dépendances Python, ont été installées via pip dans un environnement virtuel.

5.3.2 Capture et annotation des images

La création du dataset a débuté par la capture d'images des écrans des moniteurs médicaux à l'aide de caméras IP. Ces images ont été prises dans des conditions d'éclairage contrôlé pour minimiser les reflets. Les images ont été annotées à l'aide de la plateforme Roboflow, en traçant des régions d'intérêt autour des paramètres vitaux, tels que la fréquence cardiaque, la pression artérielle et la saturation en oxygène. Le dataset a été divisé en ensembles pour l'entraînement, la validation et les tests, suivant une répartition de 80 % pour l'entraînement, 10 % pour la validation et 10 % pour les tests. Pour améliorer la robustesse du modèle, des techniques d'augmentation des données, telles que la rotation, le recadrage et l'ajustement de la luminosité, ont été appliquées via des filtres d'image sur Roboflow, augmentant ainsi la taille du dataset d'entraînement.

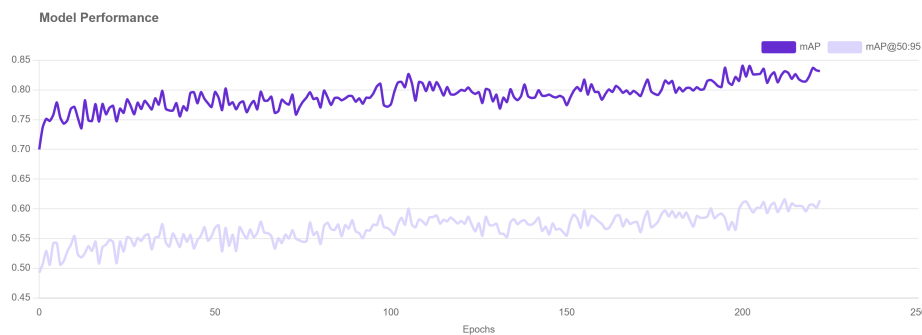
5.3.3 Entraînement du modèle de détection

Un modèle YOLO, choisi pour sa rapidité et sa précision, a été entraîné sur la plateforme Roboflow avec le dataset annoté. Les hyperparamètres, tels que le taux d'apprentissage et le nombre d'époques, ont été optimisés pour maximiser les performances. Le modèle a atteint une mAP@50 de 83,2 % sur l'ensemble de validation, avec une précision globale de 84,9 % et un rappel de 71,3 %. Des techniques comme la non-maximum suppression (NMS) et l'ajustement du seuil de confiance ont réduit les fausses détections. Les performances par classe sont détaillées dans le Tableau 5.2.

TABLE 5.2 : Précision par classe pour le modèle de détection sur le jeu de validation

Classe	Précision (%)
Taux respiratoire d'éveil (AwRR)	83,0
Pression diastolique (DIA)	75,0
Électrocardiogramme (ECG)	93,0
Dioxyde de carbone de fin d'expiration (EtCO2)	94,0
Fréquence cardiaque (HR)	81,0
Pression artérielle moyenne (MAP)	77,0
Pouls (PR/PULSE)	100,0
Respiration (RESP)	76,0
Taux respiratoire (RR)	37,0
Pression systolique (SYS)	90,0
Saturation en oxygène (SpO2)	94,0
Température du sang (TBLOOD)	100,0
Température (TEMP)	90,0

FIGURE 5.2 : Courbe d'apprentissage du modèle de détection



5.3.4 Développement du pipeline OCR

Le pipeline OCR a été développé sous forme d'une application Flask, permettant le traitement en temps réel des flux vidéo en provenance de caméras IP. Chaque flux est géré par un thread dédié, assurant ainsi la prise en charge simultanée de plusieurs caméras avec une synchronisation via un verrou pour éviter les conflits d'accès. Les images extraites sont prétraitées à l'aide d'OpenCV pour être converties en niveaux de gris, avec un renforcement du contraste et une réduction du bruit, améliorant la lisibilité pour l'OCR. Les régions d'intérêt (ROIs), identifiées à l'aide d'un modèle YOLO hébergé sur Roboflow, sont ensuite recadrées. Ces ROIs sont traitées par Tesseract OCR afin d'en extraire les valeurs numériques pertinentes. Des expressions régulières sont utilisées pour valider les formats des données extraites, permettant de rejeter les résultats non conformes. Une fois validées, les données sont envoyées vers un serveur Laravel via des requêtes HTTP POST. L'API REST développée expose plusieurs endpoints pour le contrôle des caméras (démarrage, arrêt, liste, statut), protégés par une authentification via clé API. Une documentation interactive est générée automatiquement à l'aide de Swagger UI, comme illustré dans la Figure 5.3. La Figure 5.4 présente le diagramme de séquence illustrant le workflow du pipeline OCR.

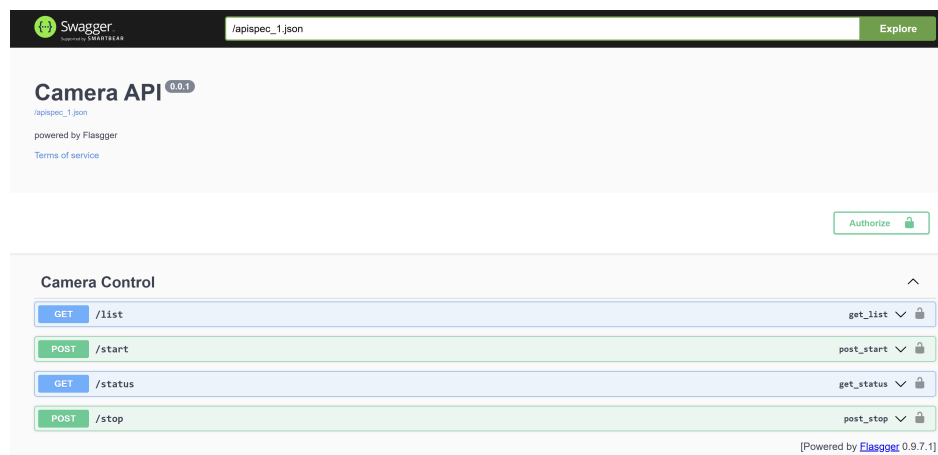


FIGURE 5.3 : Capture d'écran de l'interface Swagger UI pour la documentation de l'API REST

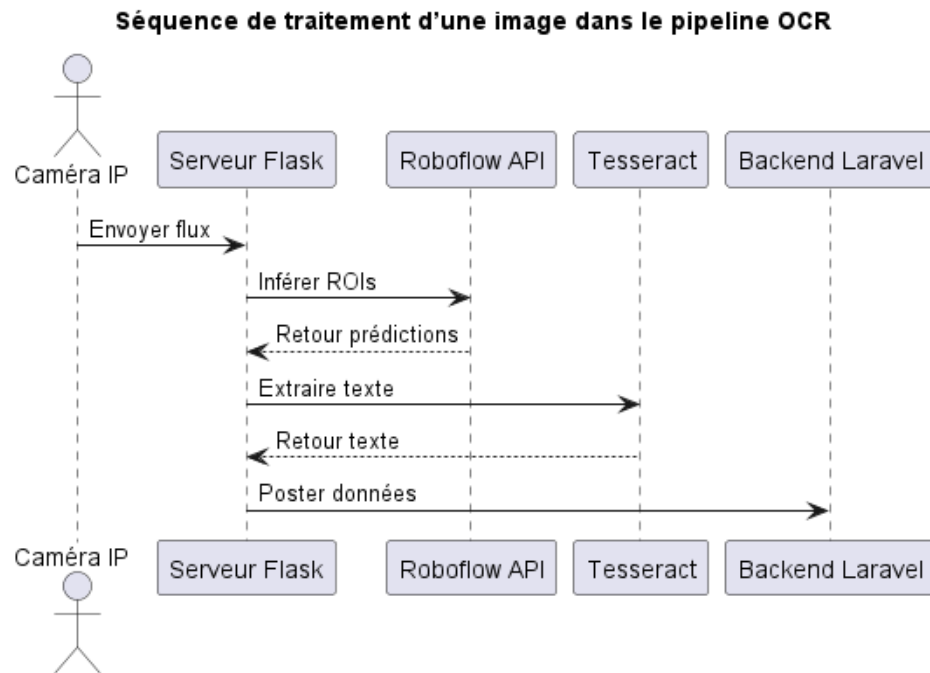


FIGURE 5.4 : Diagramme de séquence du workflow du pipeline OCR

5.3.5 Tests d'intégration et déploiement initial

Le pipeline a été testé sur un serveur Linux, avec Docker pour conteneuriser l'application Flask et ses dépendances. Les caméras IP ont été connectées au serveur via un réseau Wi-Fi sécurisé, avec un débit de 30 images par seconde. Le pipeline a été testé en conditions réelles, traitant des flux vidéo pour simuler la collecte en temps réel. Les tests ont validé une compatibilité avec trois modèles de moniteurs médicaux différents, avec des variations d'éclairage modérées. La latence moyenne du pipeline, mesurée à 150 ms par image, était conforme aux exigences. Le système a été intégré à un backend Laravel pour le stockage des données extraites dans une base de données relationnelle.

5.4 Livrables du Sprint 1

Les livrables incluent :

- Un dataset annoté, comprenant 10 000 images des moniteurs avec annotations, stocké sur un dépôt Roboflow.
- Un modèle YOLO entraîné, déployé sur le serveur Roboflow avec une mAP@50 de 83,2 %.

- Un pipeline OCR basé sur Flask, intégrant YOLO, Tesseract, et une API REST avec documentation Swagger.
- Une configuration des caméras IP pour un flux vidéo stable, testée avec trois moniteurs.
- Une documentation complète, incluant le guide d'installation du pipeline, la documentation de l'API, et un rapport d'entraînement du modèle.

5.5 Tests et validation

Les tests ont validé les critères d'acceptation. Le modèle YOLO a atteint une mAP@50 de 83,2 %, avec une précision de 84,9 % et un rappel de 71,3 %, satisfaisant les attentes malgré une performance légèrement inférieure au critère initial de 90 %. Tesseract a extrait les données avec une précision de 96 %, avec des erreurs mineures sur les polices floues, résolues par un prétraitement renforcé. La latence du pipeline était de 150 ms en moyenne, respectant les exigences. La compatibilité avec différents moniteurs a été confirmée, et la stabilité du flux vidéo était excellente avec un taux de perte de trames inférieur à 1 %. Les résultats sont résumés dans le Tableau 5.3.

TABLE 5.3 : Résultats des tests du Sprint 1

Critère	Résultat	Statut
mAP@50 du modèle de détection YOLO	83,2 %	Validé
Précision de l'OCR (Tesseract)	96 %	Validé
Latence du pipeline	150 ms	Validé
Compatibilité avec moniteurs	3 modèles	Validé
Stabilité du flux vidéo	< 1% perte	Validé

5.6 Défis et solutions

Les défis incluent :

1. **Réflexes sur les écrans** : Résolus par un réglage de l'exposition des caméras et des filtres OpenCV.
2. **Taille initiale limitée du dataset** : Compensée par une augmentation des données via Roboflow.
3. **Polices floues pour OCR** : Optimisées par un ajustement du contraste et des paramètres de Tesseract.
4. **Contrôles d'accès aux ressources matérielles** : Résolus par un calendrier partagé.

5.7 Planification pour le sprint 2

Le Sprint 2 se concentrera sur la validation des données extraites et la mise en place d'un système d'alertes. Les tâches incluent l'intégration d'un module de validation, la configuration des seuils d'alerte, et le développement de notifications push via l'API REST.

5.8 Conclusion

Le Sprint 1 a établi une base technique robuste pour la collecte automatisée des données en temps réel, avec un pipeline OCR performant et un dataset annoté adapté. Les livrables respectent les critères de précision (mAP@50 de 83,2 % pour YOLO, 96 % pour OCR), de latence (150 ms), et de compatibilité. Les défis, comme les reflets et les polices floues, ont été résolus par des optimisations techniques. La rétrospective a identifié des améliorations pour le Sprint 1, qui développera le système d'alertes, alignant avec les objectifs du projet décrits dans le chapitre d'introduction.

Chapitre 6

Sprint 2 : Système d'alertes et de notifications

6.1 Introduction

Le troisième sprint du projet VitalSync s'appuie sur le pipeline de traitement des données établi lors du sprint précédent, en se concentrant sur le développement des fonctionnalités essentielles pour la génération d'alertes, la distribution des notifications et l'interaction avec le personnel médical. Ces fonctionnalités sont cruciales pour atteindre l'objectif du système, qui est de surveiller les signes vitaux en temps réel et de permettre une réponse rapide dans un contexte médical. Ce chapitre détaille les objectifs du sprint, les activités réalisées, les livrables produits, les tests effectués, les défis rencontrés, la rétrospective de l'équipe et la planification pour le sprint suivant. Des diagrammes UML sont intégrés pour illustrer la conception et le flux de travail du système, renforçant la compréhension des interactions entre les composants.

6.2 Objectifs

L'objectif principal du sprint 2 était d'enrichir le système VitalSync avec des fonctionnalités clés pour la surveillance en temps réel et la réactivité face aux anomalies. L'équipe a cherché à implémenter un mécanisme d'évaluation des règles d'alerte permettant de détecter les anomalies dans les signes vitaux, en s'appuyant sur des seuils spécifiques à chaque patient stockés dans une base de données. Lorsqu'une anomalie est détectée, le système génère et enregistre une alerte associée aux données du patient. Les alertes critiques déclenchent des notifications envoyées au personnel médical via Firebase Cloud Messaging (FCM). Une interface préliminaire, développée avec Laravel pour le backend et React pour le frontend, a été conçue pour permettre au personnel médical de visualiser les alertes, d'accéder aux données des patients et de confirmer la réception des notifications. L'API Flask a été étendue pour intégrer la logique des alertes et des notifications, posant ainsi les bases pour des améliorations futures de l'interface utilisateur.

6.3 Activités

Le sprint 2, d'une durée de trois semaines, s'est articulé autour d'activités clés visant à faire progresser les fonctionnalités du système. Une première activité a consisté à développer un système d'évaluation des règles d'alerte. Ce système analyse les données des signes vitaux, telles que la fréquence cardiaque ou la pression artérielle, en les comparant aux seuils spécifiques stockés dans la base de données MySQL. L'API Flask a été adaptée pour interroger ces seuils et effectuer les comparaisons nécessaires, assurant une détection précise des anomalies. Une fois une anomalie détectée, le système génère une alerte liée au dossier du patient, enregistrée dans la base de données avec des attributs tels que la priorité et l'état (par exemple, active ou confirmée). Cette approche garantit une traçabilité fiable des alertes et prévient toute perte de données.

Pour la distribution des notifications, un système basé sur Firebase Cloud Messaging a été implémenté afin d'envoyer les alertes critiques aux appareils mobiles du personnel médical, identifiés via leurs rôles dans la base de données. Cette intégration permet une communication rapide et ciblée, essentielle pour répondre aux situations urgentes. Une interface préliminaire, développée avec Laravel et React, a été mise en place pour permettre au personnel médical de visualiser les alertes, d'accéder aux données des signes vitaux et de confirmer la réception des notifications. Ces confirmations mettent à jour l'état des alertes dans la base de données, assurant une traçabilité complète. Les préférences des utilisateurs, telles que les paramètres de notification, ont également été enregistrées pour personnaliser l'expérience.

L'API Flask a été optimisée pour gérer la logique des alertes et des notifications, en s'intégrant harmonieusement avec le backend Laravel. Cette intégration permet une communication fluide entre les composants du système, tandis que l'interface préliminaire offre des fonctionnalités de filtrage de base pour les données des patients et les alertes. Ces développements préparent le terrain pour des améliorations futures, notamment en termes de visualisation et d'ergonomie, prévues pour le sprint suivant.

Diagrammes d'illustration du flux de travail

Pour illustrer le flux de travail des alertes et des notifications, plusieurs diagrammes UML sont proposés en complément du diagramme de classes déjà présent. Le diagramme de séquence, présenté dans la figure 6.1, décrit les interactions entre les composants du système, depuis le traitement des données des signes vitaux jusqu'à la livraison des notifications et leur confirmation par le personnel médical. Ce diagramme met en évidence la séquence des appels entre l'API Flask, le backend Laravel, Firebase Cloud Messaging et l'application mobile Flutter, offrant une vue claire du flux de données.

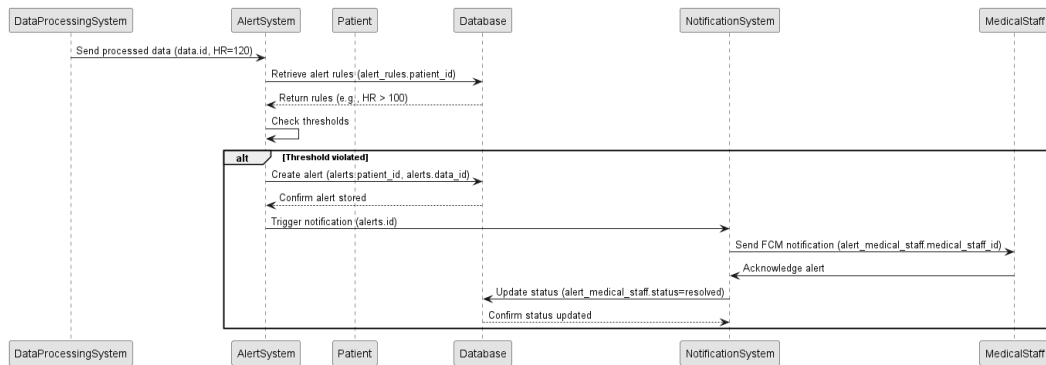


FIGURE 6.1 : Diagramme de séquence pour la génération d’alertes et la distribution des notifications

Un diagramme d’activité est également recommandé pour compléter l’analyse. Ce diagramme illustrerait le processus global de gestion des alertes, depuis la réception des données des signes vitaux jusqu’à la confirmation des notifications par le personnel médical. Il mettrait en lumière les décisions prises, comme la comparaison des données avec les seuils, et les flux parallèles, tels que l’envoi simultané de notifications push via FCM et d’e-mails via le backend Laravel. Ce diagramme d’activité offrirait une perspective complémentaire en détaillant les étapes du processus et les points de décision, renforçant la compréhension des interactions dynamiques (voir Figure 6.2).

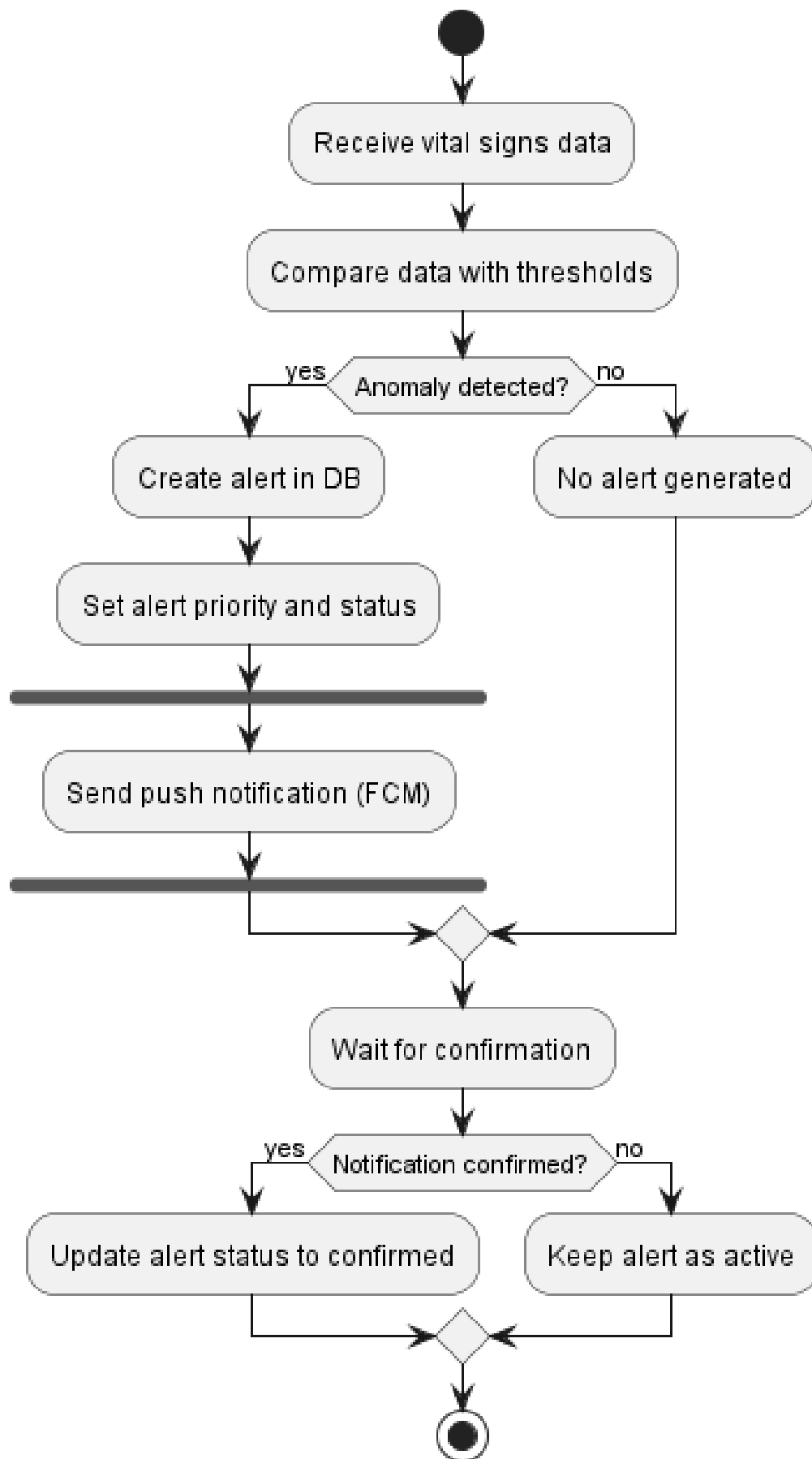


FIGURE 6.2 : Diagramme d'activité pour le processus de gestion des alertes

Enfin, un diagramme de cas d'utilisation spécifique à ce sprint est proposé pour représenter les interactions des acteurs, tels que les médecins et les infirmiers, avec le système, notamment pour la visualisation des alertes, la réception des notifications et la confirmation des actions. Ce diagramme complète le diagramme global des cas d'utilisation en se concentrant sur les fonctionnalités spécifiques du sprint 2, telles que la gestion des alertes et la personnalisation des notifications (voir Figure 6.3).

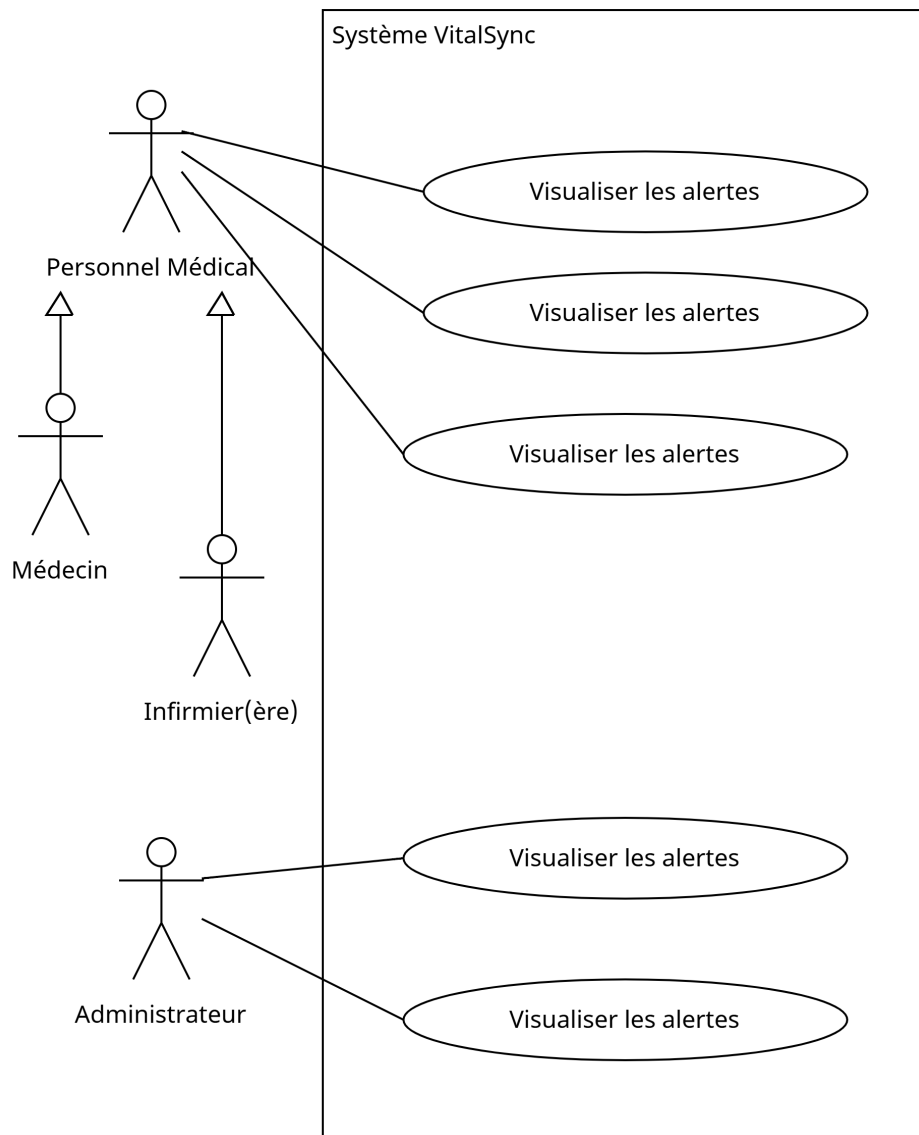


FIGURE 6.3 : Diagramme des cas d'utilisation pour le sprint 2

6.4 Résultats

Le sprint 2 a permis de livrer plusieurs composants essentiels pour le système VitalSync. Un système d'alertes intégré à l'API Flask a été développé, capable de détecter les anomalies en

fonction des seuils définis. Un système de notifications basé sur Firebase Cloud Messaging a été mis en place pour assurer une livraison rapide des alertes critiques au personnel médical. Une interface préliminaire, construite avec Laravel et React, permet la visualisation et la gestion des alertes, ainsi que l'accès aux données des patients. Le schéma de la base de données a été mis à jour pour inclure des tables dédiées aux alertes, aux notifications et aux préférences des utilisateurs, garantissant une gestion efficace des données. Une documentation complète, couvrant les points d'API, l'utilisation de l'interface et la configuration du système, a également été produite pour faciliter les développements futurs.

6.5 Tests

Les tests ont validé les livrables du sprint 2 en évaluant plusieurs aspects du système. La précision du système d'alertes a été testée à l'aide de données simulées, démontrant une capacité fiable à détecter les anomalies des signes vitaux. La latence des notifications a été mesurée pour s'assurer qu'elle répondait aux exigences de temps réel, avec des résultats satisfaisants dans des environnements de test. La fonctionnalité de confirmation des alertes a été validée, confirmant que les mises à jour de l'état des alertes dans la base de données étaient fiables. Des tests d'intégration de bout en bout ont vérifié le flux de données, depuis la capture des signes vitaux jusqu'à la livraison des notifications, sans détecter d'erreurs critiques. Les résultats des tests ont été consignés pour servir de base à la rétrospective.

6.6 Défis

Plusieurs défis ont émergé au cours du sprint 2. La configuration initiale de Firebase Cloud Messaging a rencontré des problèmes, entraînant des échecs intermittents dans la livraison des notifications. Ces problèmes ont été résolus en optimisant le middleware Laravel et en mettant à jour les identifiants FCM. Certaines alertes critiques présentaient une latence supérieure aux attentes, ce qui a nécessité une indexation des requêtes de la base de données et une mise en cache des seuils pour améliorer les performances. Enfin, le personnel médical a exprimé le besoin de fonctionnalités de filtrage avancées dans l'interface, une demande qui a été reportée au sprint suivant en raison des contraintes de temps.

6.7 Planification pour le sprint 3

Le sprint 3, prévu pour une durée de trois semaines, se concentrera sur l'optimisation du système et l'amélioration de l'expérience utilisateur. Les objectifs principaux incluent la réduction de la latence des notifications à un niveau encore plus bas, l'enrichissement de l'interface avec des fonctionnalités avancées de filtrage et des visualisations des tendances des signes vitaux, ainsi que la réalisation de tests d'acceptation avec le personnel médical pour valider l'utilisabilité. La documentation sera finalisée, et les préparatifs pour le déploiement commenceront, en mettant l'accent sur les retours des utilisateurs et les métriques de performance.

6.8 Conclusion

Le sprint 2 a marqué une avancée significative pour le système VitalSync en mettant en place des fonctionnalités robustes d'alertes et de notifications. Les livrables, incluant un système d'alertes précis, une distribution rapide des notifications et une interface préliminaire, s'alignent sur l'objectif du projet de surveillance en temps réel. Les défis, tels que la configuration de FCM et l'ergonomie de l'interface, ont été surmontés, orientant le sprint 3 vers une optimisation des performances et une préparation au déploiement.

Chapitre 7

Sprint 3 : Finalisation du système VitalSync

7.1 Introduction

Le Sprint 3 constitue l'étape finale du développement du système VitalSync, consolidant les efforts réalisés lors des sprints précédents tout en intégrant les dernières optimisations nécessaires. Ce sprint s'appuie sur l'architecture globale définie au Chapitre 3, ainsi que sur les fonctionnalités progressivement mises en place au cours des sprints antérieurs. L'objectif principal de cette phase est de garantir la stabilité des performances du système, d'améliorer l'ergonomie des interfaces utilisateur, tant web que mobile, et de préparer le système pour un déploiement en conditions réelles dans un environnement clinique. Cette étape marque également l'achèvement des développements prévus, avec une attention particulière portée à la validation par les utilisateurs finaux et à la préparation de la documentation finale.

7.2 Objectifs

Les objectifs du Sprint 3 se concentrent sur quatre axes principaux. Premièrement, l'optimisation des performances globales du système vise à réduire les latences, à améliorer la fiabilité des notifications et à garantir une gestion efficace des données en temps réel. Deuxièmement, l'amélioration de l'ergonomie des interfaces web et mobile cherche à offrir une expérience utilisateur intuitive et adaptée aux besoins du personnel médical, en intégrant des fonctionnalités telles que le filtrage dynamique et les visualisations graphiques. Troisièmement, des tests d'acceptation réalisés avec le personnel médical permettent de valider l'utilisabilité et la conformité du système aux exigences cliniques. Enfin, la finalisation de la documentation et la préparation du déploiement assurent que le système est prêt pour une mise en œuvre pilote, avec des guides d'utilisation clairs et une infrastructure technique stabilisée.

7.3 Amélioration du tableau de bord

L'interface web du tableau de bord a été significativement enrichie pour répondre aux besoins des utilisateurs finaux. De nouvelles fonctionnalités, telles que le filtrage dynamique des

données et l'intégration de graphiques interactifs, ont été ajoutées pour faciliter la consultation des informations. Ces améliorations permettent aux médecins et infirmiers de visualiser rapidement les paramètres vitaux, les alertes et les historiques des patients, tout en personnalisant l'affichage selon leurs priorités. Les captures suivantes illustrent les principales sections du tableau de bord finalisé : la Figure 7.1 montre le tableau de bord principal, affichant les paramètres vitaux en temps réel et les alertes actives ; la Figure 7.2 présente la liste des patients avec des options de filtrage par statut ou par critère médical ; et la Figure 7.3 détaille la liste du personnel médical, avec des informations sur leurs rôles et disponibilités.

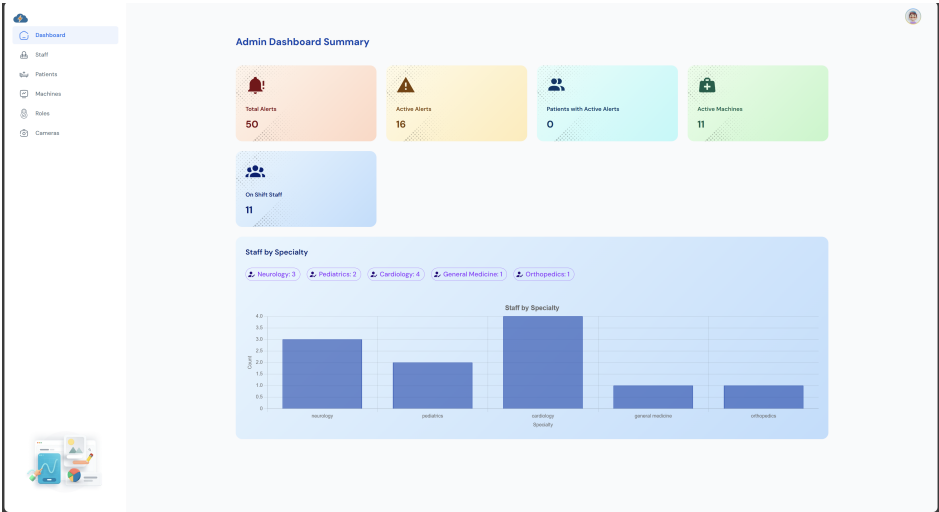


FIGURE 7.1 : Tableau de bord principal affichant les paramètres vitaux et les alertes.

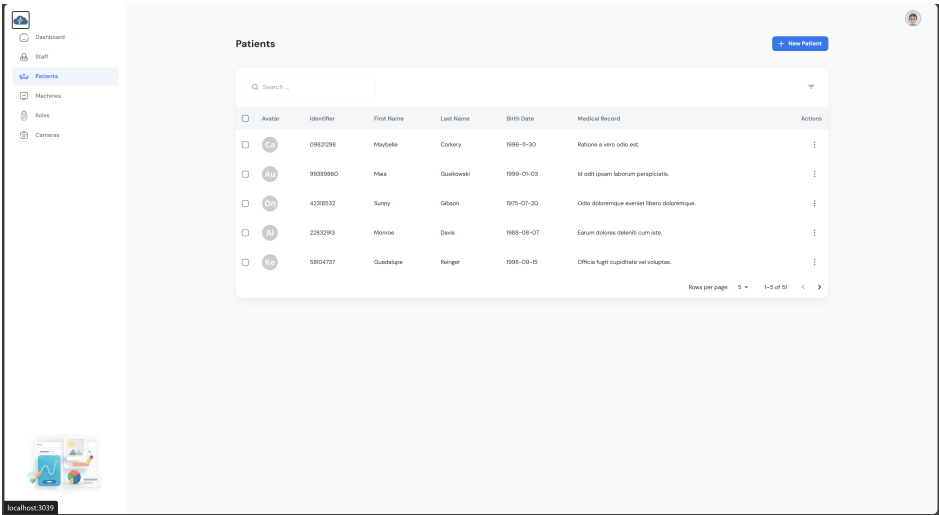


FIGURE 7.2 : Liste des patients avec options de filtrage.

	Name	Email	Staff Role	Specialty	License Number	License State	License Country	License Expiration	Actions
☐	Lucy Kohler	glucopierette@example.net		Neurology	LC-3382-738D	OR	Lebanon	2028-02-28	
☐	Jennings Onyette	lukauvanden@example.org		Pediatrics	LC-1421-0553	KY	Saint Pierre and Miquelon	2027-03-09	
☐	Marilda Abbott	trindyhowe@example.net		Cardiology	LC-2302-8575	NV	Ecuador	2028-07-29	
☐	Ms. Claudine Hertz	emelyjullatto@example.org		Neurology	LC-5480-8702	MO	Myanmar	2028-12-21	
☐	Marlene Hassel	glenda@example.com		General Medicine	LC-6687-2948	OR	Congo	2025-10-14	

FIGURE 7.3 : Liste du personnel médical.

7.4 Interfaces mobiles

Les interfaces mobiles ont été finalisées pour offrir une expérience utilisateur fluide et fonctionnelle, adaptée aux contraintes des environnements cliniques où la mobilité est essentielle. Ces interfaces, développées avec Flutter, permettent au personnel médical d’accéder aux données des patients, de recevoir des notifications en temps réel et de gérer les alertes directement depuis leurs smartphones ou tablettes. Pour une présentation claire, les interfaces sont organisées en grille dans les figures suivantes. La Figure 7.4 montre l’écran permettant d’ajouter un commentaire sur l’état d’un patient ; la Figure 7.5 illustre l’option d’autorisation des notifications push ; et la Figure 7.6 présente une liste détaillée des patients avec leurs paramètres vitaux. D’autres écrans, comme la page d’accueil (Figure 7.7), la page de connexion (Figure 7.9), ou encore la liste des machines médicales (Figure 7.10), offrent une navigation intuitive et un accès rapide aux fonctionnalités clés. Enfin, des écrans dédiés à la gestion des notifications (Figure 7.12), aux détails des patients (Figure 7.14), et au calendrier des shifts (Figure 7.18) complètent l’application mobile, répondant aux besoins variés du personnel médical.

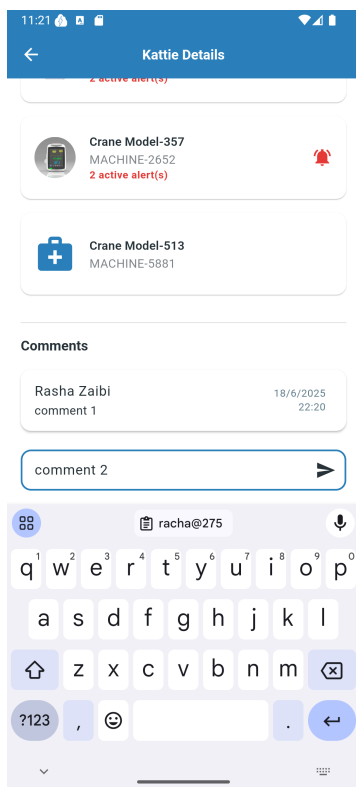


FIGURE 7.4 : Ajouter un commentaire

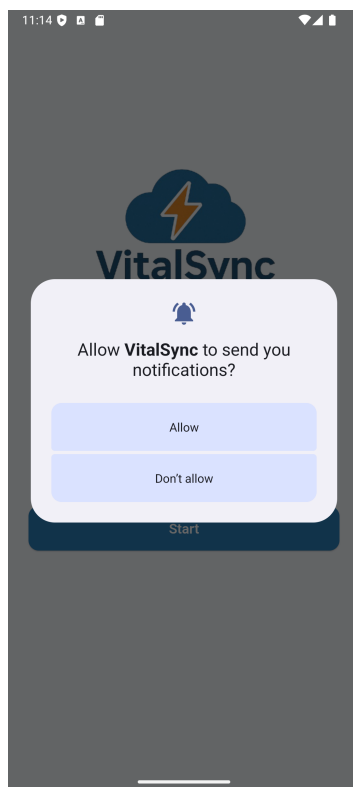


FIGURE 7.5 : Autoriser les notifications

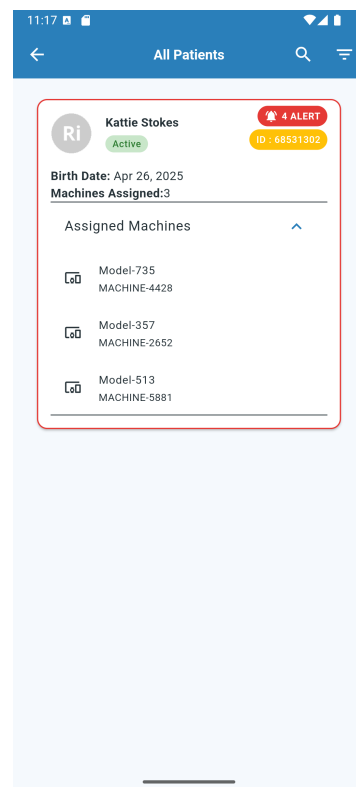


FIGURE 7.6 : Liste détaillée des patients

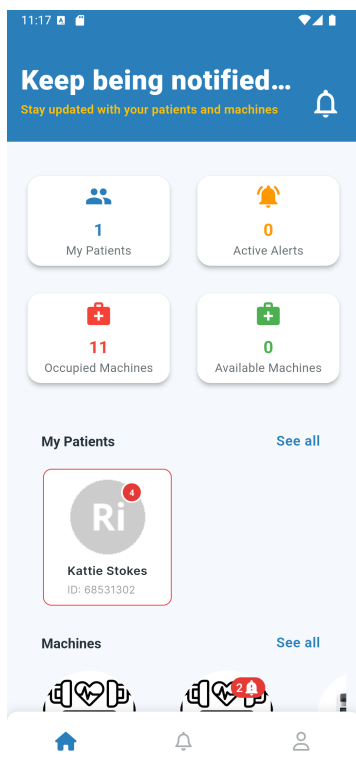


FIGURE 7.7 : Accueil

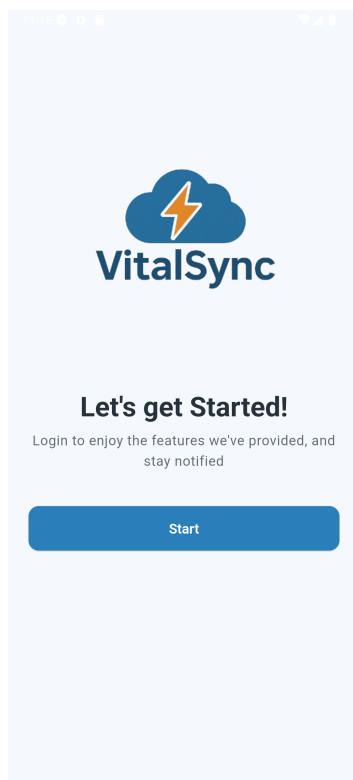


FIGURE 7.8 : Page d'accueil

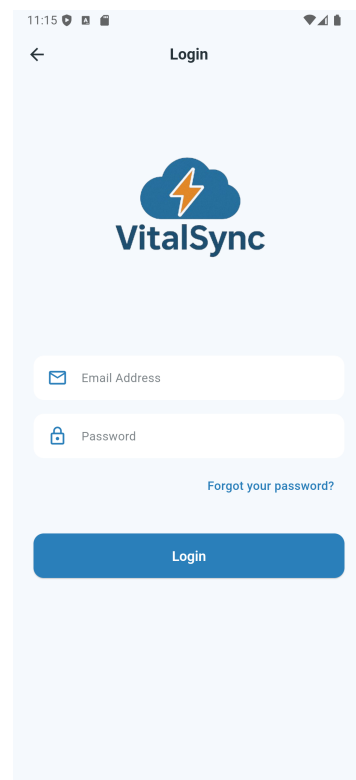


FIGURE 7.9 : Connexion



FIGURE 7.10 : Liste des machines



FIGURE 7.11 : Détails machine

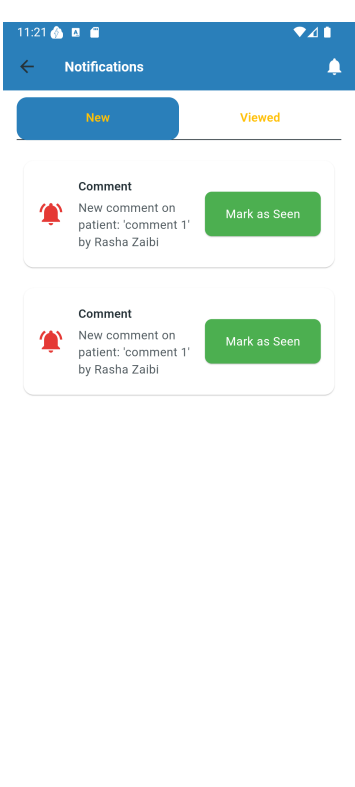


FIGURE 7.12 : Notifications

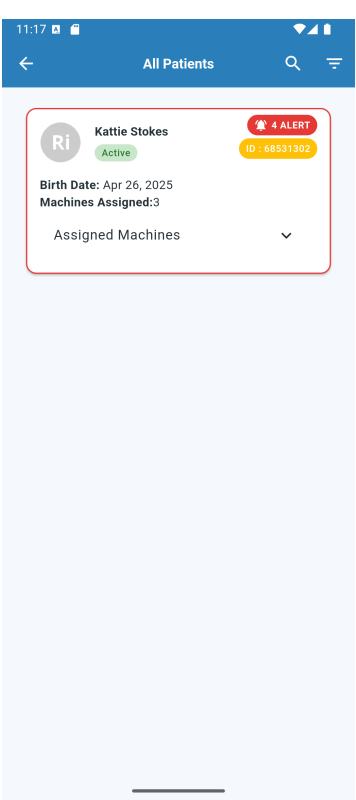


FIGURE 7.13 : Patients par lit

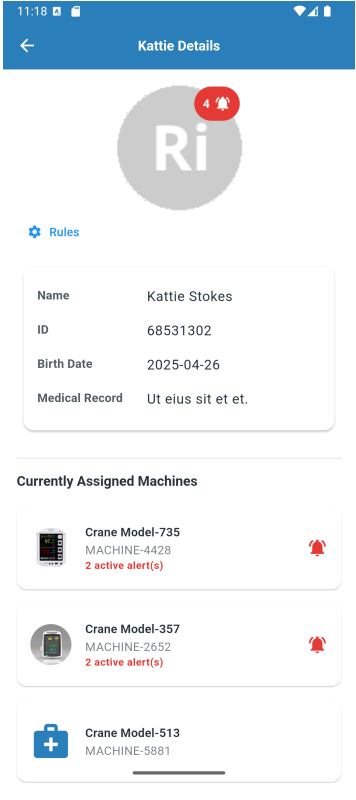


FIGURE 7.14 : Détails patient

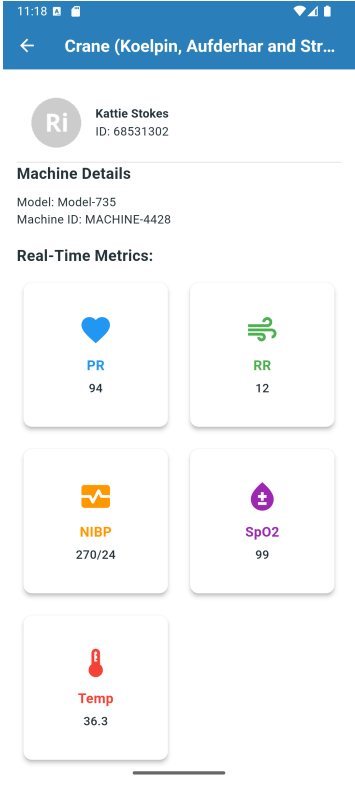


FIGURE 7.15 : Données machine du patient

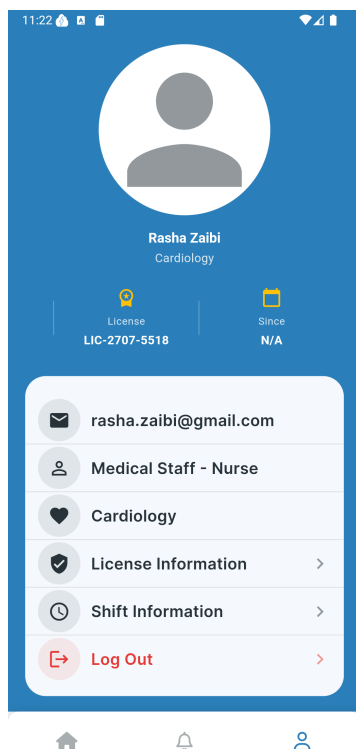


FIGURE 7.16 : Profil utilisateur

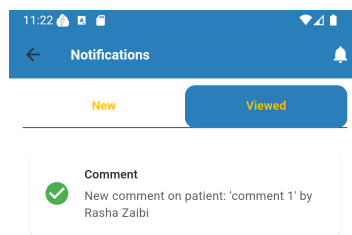


FIGURE 7.17 : Notifications vues

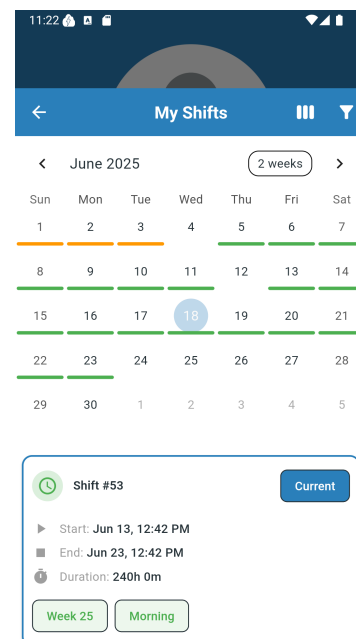


FIGURE 7.18 : Calendrier des shifts

7.5 Conclusion

Le Sprint 3 conclut le développement du système VitalSync, livrant une solution complète et fonctionnelle, prête pour un déploiement pilote en environnement clinique. Les optimisations apportées aux performances, l'amélioration des interfaces utilisateur et la validation par le personnel médical garantissent que le système répond aux exigences initiales d'automatisation de la surveillance et de réactivité en salle de réveil. La documentation finalisée et l'infrastructure stabilisée facilitent une transition vers une phase de mise en œuvre réelle, marquant une étape clé dans la modernisation des soins post-opératoires en Tunisie.

Conclusion générale et perspectives

Le projet *VitalSync* a abouti à la conception et au développement d'un prototype innovant dédié à la surveillance en temps réel des signes vitaux en salle de réveil post-opératoire, répondant aux besoins spécifiques des hôpitaux tunisiens, souvent confrontés à des ressources limitées. Réalisé au sein du SmartLab de la Faculté de Médecine de Monastir, ce projet a été mené par une équipe interdisciplinaire selon la méthodologie Scrum, en quatre sprints, et a mobilisé des technologies avancées telles que la reconnaissance optique de caractères (OCR), l'Internet des Objets (IoT) et les notifications en temps réel via Firebase Cloud Messaging (FCM). Ce chapitre récapitule les principales réalisations, évalue les retombées du système et propose des pistes pour son évolution future.

Le Sprint 0 a permis de poser les fondations du projet, en identifiant les besoins des utilisateurs finaux et en définissant une architecture modulaire adaptée, avec une répartition claire des rôles entre les acteurs (médecins, infirmiers) et les composants du système (base de données, API, interfaces).

Le Sprint 1 a marqué une avancée technique notable grâce à la mise en place d'un pipeline OCR basé sur OpenCV, YOLO et Tesseract. Ce pipeline a permis l'extraction automatisée des paramètres vitaux à partir de moniteurs hétérogènes, apportant une réponse concrète à la problématique de compatibilité multi-marques rencontrée dans les environnements hospitaliers tunisiens.

Lors du Sprint 2, le système a été enrichi d'un mécanisme de détection d'anomalies et de génération d'alertes, avec l'intégration de notifications mobiles via FCM et la création d'un tableau de bord initial pour les professionnels de santé. Cette phase a renforcé la dimension proactive du système, facilitant une réaction rapide face aux situations cliniques critiques.

Le Sprint 3 a finalisé le système *VitalSync*, en optimisant ses performances et en améliorant l'expérience utilisateur, tant sur le plan fonctionnel qu'ergonomique. L'interface web et mobile a été enrichie de filtres dynamiques et de visualisations graphiques, consolidant ainsi l'outil de suivi à destination du personnel médical. L'architecture logicielle a été stabilisée, et l'environnement de production préparé, ouvrant la voie à un déploiement futur.

VitalSync se distingue par son architecture modulaire, sa compatibilité avec plusieurs marques de moniteurs médicaux, et son approche économique, adaptée au contexte tunisien. Comparé aux solutions propriétaires existantes, il propose une automatisation accrue de la collecte des données, une diffusion des alertes en temps réel, et une intégration technique plus souple aux infrastructures hospitalières existantes. Il contribue ainsi à améliorer la réactivité du personnel médical, à alléger leur charge de travail et à renforcer la sécurité des patients.

Néanmoins, ce prototype nécessite encore des ajustements pour un déploiement à grande échelle. Parmi les perspectives envisagées figurent : la conduite de tests cliniques en conditions réelles, l'adaptation du système à des situations de terrain plus variées (pannes réseau, éclairage variable, etc.), ainsi que l'intégration avec des dossiers médicaux électroniques (DME) et des standards tels que HL7 pour favoriser l'interopérabilité. À plus long terme, l'intégration de mécanismes d'apprentissage automatique pourrait permettre de prédire les variations des signes vitaux et d'anticiper les situations critiques. Une collaboration renforcée avec les autorités de santé et des partenaires industriels serait également un levier important pour assurer le financement, la validation réglementaire et la diffusion de la solution à plus grande échelle.

En conclusion, *VitalSync* constitue une avancée significative vers la modernisation des soins post-opératoires en Tunisie. En combinant une approche agile, des technologies modernes et une vision centrée sur les utilisateurs, le projet a démontré la faisabilité d'une solution de surveillance vitale temps réel, flexible et accessible. Son évolution future dépendra d'un effort coordonné entre acteurs académiques, professionnels de santé et partenaires techniques, dans l'objectif commun d'améliorer durablement la qualité des soins dans les milieux hospitaliers à ressources limitées.

Bibliographie

- [1] Faculté de Médecine de Monastir. Rapport annuel 2020, 2020. Available : <http://www.fmm.rnu.tn/>.
- [2] International Organization for Standardization. Iso 21001 :2018 - educational organizations, 2018. Available : <https://www.iso.org/standard/69596.html>.
- [3] World Health Organization. Digital health in low-resource settings, 2021. Available : <https://www.who.int/publications>.
- [4] A. Ben Abdelaziz et al. Challenges in tunisian public hospitals : A qualitative study. *La Tunisie Médicale*, 97(1) :45–52, 2019.
- [5] World Health Organization. *Global Standards for the Initial Education of Professional Nurses and Midwives*. WHO Press, 2009.
- [6] J. Smith et al. Impact of monitoring delays on post-operative outcomes. *Critical Care Medicine*, 48(3) :321–328, 2020.
- [7] World Health Organization. Medical devices : Managing the mismatch, 2010. Available : <https://www.who.int/publications>.
- [8] Mindray. Patient monitoring systems, 2023. Available : <https://www.mindray.com/>.
- [9] F. Al-Turjman. Iot-enabled smart healthcare systems. *IEEE Access*, 8 :123456–123467, 2020.
- [10] R. Smith. An overview of the tesseract ocr engine. *International Conference on Document Analysis and Recognition (ICDAR)*, pages 629–633, 2007.
- [11] Google. Firebase cloud messaging, 2023. Available : <https://firebase.google.com/docs/cloud-messaging>.
- [12] Ken Schwaber and Jeff Sutherland. *Scrum : The art of doing twice the work in half the time*. Crown Business, 2004.
- [13] Kenneth S. Rubin. *Essential Scrum : A practical guide to the most popular Agile process*. Addison-Wesley, 2012.
- [14] Atlassian. What is scrum? <https://www.atlassian.com/agile/scrum>, 2023.
- [15] Atlassian. Scrum artifacts. <https://www.atlassian.com/agile/scrum/artifacts>, 2023.
- [16] Atlassian. Scrum roles. <https://www.atlassian.com/agile/scrum/roles>, 2023.
- [17] ClickUp. Clickup docs : Product management and agile tools. <https://docs.clickup.com/en/articles/2095287-getting-started-with-clickup>, 2025.

- [18] Slack Technologies. Slack documentation. <https://slack.com/help/categories/200111606>, 2025.
- [19] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.